

Notas matemáticas complementarias del proyecto
nicolas-lm

Contents

1	Teoría de la probabilidad	3
1.1	Sigma-álgebras, topologías y medidas	3
1.2	Variables aleatorias	4
1.2.1	Medibilidad de funciones	4
1.2.2	Distribución y esperanza	5
1.3	Condicionales, marginales e independencia	9
1.3.1	Distribución conjunta y marginal	9
1.3.2	Independencia	9
1.3.3	Distribuciones condicionales	10
1.4	Desigualdades de concentración	12
2	Aprendizaje estadístico	14
2.1	Espacio de datos y distribución poblacional	14
2.2	Hipótesis y clases de hipótesis	15
2.3	Funciones de pérdida	15
2.4	Riesgo poblacional	15
2.5	Riesgo de Bayes y error de aproximación	16
2.6	Riesgo empírico	16
2.7	Minimización del riesgo empírico	17
2.8	Generalización	17
2.9	Descomposición básica del exceso de riesgo	18
2.10	Sobreajuste y regularización	19
2.11	Clasificación y regresión	19
2.12	Entropía cruzada y modelos probabilísticos	19
3	Redes neuronales	21
3.1	Capas, activaciones y parámetros	21
3.2	Redes neuronales feedforward	22
3.3	Preactivaciones y activaciones	23
3.4	Número de parámetros	23
3.5	La red como familia paramétrica de funciones	24
3.6	Interpretación geométrica	24
3.7	Observación sobre profundidad y expresividad	24
4	Modelos de lenguaje implementados en nicolas-lm	25
4.1	Modelado autorregresivo del lenguaje	25
4.2	Modelo bigrama	26
4.3	Transformer causal pequeño	28
4.3.1	Embeddings de token y posición	29
4.3.2	Autoatención causal	29

4.3.3	Bloque Transformer pre-norm	32
4.4	Modelo estilo LLaMA	32
4.4.1	RMSNorm	33
4.4.2	Embeddings posicionales rotatorios	33
4.4.3	Atención causal estilo LLaMA	35
4.4.4	SwiGLU	35
4.4.5	Bloque Transformer estilo LLaMA	36
4.4.6	Por qué estas modificaciones son estilo LLaMA	37
4.5	Comparación de los tres modelos	38
4.6	Pérdida común de entrenamiento	38
4.7	Generación autorregresiva	38
4.8	Interpretación dentro del proyecto	39
5	Entrenamiento de redes neuronales	40
5.1	Descenso de gradiente	40
5.1.1	Condiciones estructurales y existencia de minimizadores	41
5.1.2	Convergencia del descenso de gradiente	44
5.2	Descenso de gradiente estocástico	45
5.2.1	Motivación y tasas de aprendizaje decrecientes	45
5.2.2	Convergencia del descenso de gradiente estocástico	47
5.3	Tasas de aprendizaje adaptativas y por coordenadas	49
5.3.1	Escalamiento por coordenadas	49
5.3.2	Algoritmos	50
5.4	Retropropagación	53
5.4.1	Idea básica	54
5.4.2	Redes neuronales feedforward	55

Chapter 1

Teoría de la probabilidad

Estas notas presentan los fundamentos matemáticos del proyecto `nicolas-lm`. Comenzamos con teoría de la probabilidad (Capítulo 1), continuamos con aprendizaje estadístico (Capítulo 2) y redes neuronales feedforward (Capítulo 3), aplicamos lo anterior a los modelos autorregresivos implementados en el código (Capítulo 4) y cerramos con métodos de entrenamiento basados en descenso de gradiente (Capítulo 5). La presentación combina material de referencia estándar; las fuentes principales son los textos de Petersen and Zech [27] y Calin [8], complementados puntualmente con Wegner [35].

1.1 Sigma-álgebras, topologías y medidas

Sea Ω un conjunto, y denotemos por 2^Ω el conjunto potencia de Ω .

Definición 1.1.1. Un subconjunto $\mathcal{A} \subseteq 2^\Omega$ se llama una **sigma-álgebra**¹ sobre Ω si satisface

- (i) $\Omega \in \mathcal{A}$,
- (ii) $A^c \in \mathcal{A}$ siempre que $A \in \mathcal{A}$,
- (iii) $\bigcup_{i \in \mathbb{N}} A_i \in \mathcal{A}$ siempre que $A_i \in \mathcal{A}$ para todo $i \in \mathbb{N}$.

Para una sigma-álgebra $\mathcal{A}$ sobre Ω , la pareja $(\Omega, \mathcal{A})$ también se denomina un **espacio medible**. Para un espacio medible, un subconjunto $A \subseteq \Omega$ se llama **medible** si $A \in \mathcal{A}$. Los conjuntos medibles también se llaman **eventos**.

Otro sistema clave de subconjuntos de Ω es el de una topología.

Definición 1.1.2. Un subconjunto $\mathcal{T} \subseteq 2^\Omega$ se llama una **topología** sobre Ω si satisface

- (i) $\emptyset, \Omega \in \mathcal{T}$,
- (ii) $\bigcap_{j=1}^n O_j \in \mathcal{T}$ siempre que $n \in \mathbb{N}$ y $O_1, \dots, O_n \in \mathcal{T}$,
- (iii) $\bigcup_{i \in I} O_i \in \mathcal{T}$ siempre que para un conjunto de índices I se tenga $O_i \in \mathcal{T}$ para todo $i \in I$.

Si $\mathcal{T}$ es una topología sobre Ω , llamamos a $(\Omega, \mathcal{T})$ un **espacio topológico**, y un conjunto $O \subseteq \Omega$ se llama **abierto** si y solo si $O \in \mathcal{T}$.

¹Usamos esta notación en lugar de la más común “ σ -álgebra” para evitar confusión con la función de activación σ .

Observación 1.1.3. Las dos nociones difieren en que una topología permite uniones de conjuntos arbitrarias, posiblemente no numerables, pero únicamente intersecciones finitas, mientras que una sigma-álgebra permite uniones e intersecciones numerables.

Ejemplo 1.1.4. Sea $d \in \mathbb{N}$ y denotemos por $B_\varepsilon(x) = \{y \in \mathbb{R}^d \mid \|y - x\| < \varepsilon\}$ el conjunto de puntos cuya distancia euclidiana a x es menor que ε . Entonces, para todo $A \subseteq \mathbb{R}^d$, la topología más pequeña sobre A que contiene $A \cap B_\varepsilon(x)$ para todo $\varepsilon > 0$, $x \in \mathbb{R}^d$, se llama la **topología euclidiana** sobre A .

Si $(\Omega, \mathcal{T})$ es un espacio topológico, entonces la **sigma-álgebra de Borel** se refiere a la sigma-álgebra más pequeña sobre Ω que contiene todos los conjuntos abiertos, es decir, todos los elementos de $\mathcal{T}$. A lo largo de este libro, los subconjuntos de $\mathbb{R}^d$ siempre se entienden equipados con la topología euclidiana. La sigma-álgebra de Borel sobre $\mathbb{R}^d$ se denota por $\mathcal{B}_d$.

Ahora podemos introducir las medidas.

Definición 1.1.5. Sea $(\Omega, \mathcal{A})$ un espacio medible. Una aplicación $\mu : \mathcal{A} \rightarrow [0, \infty]$ se llama una **medida** si satisface

- (i) $\mu(\emptyset) = 0$,
- (ii) para toda sucesión $(A_i)_{i \in \mathbb{N}} \subseteq \mathcal{A}$ tal que $A_i \cap A_j = \emptyset$ siempre que $i \neq j$, se tiene

$$\mu\left(\bigcup_{i \in \mathbb{N}} A_i\right) = \sum_{i \in \mathbb{N}} \mu(A_i).$$

Decimos que la medida es **finita** si $\mu(\Omega) < \infty$, y que es **sigma-finita** si existe una sucesión $(A_i)_{i \in \mathbb{N}} \subseteq \mathcal{A}$ tal que $\Omega = \bigcup_{i \in \mathbb{N}} A_i$ y $\mu(A_i) < 1$ para todo $i \in \mathbb{N}$. En caso de que $\mu(\Omega) = 1$, la medida se llama **medida de probabilidad**.

Ejemplo 1.1.6. Se puede mostrar que existe una única medida λ sobre $(\mathbb{R}^d, \mathcal{B}_d)$ tal que, para todos los conjuntos de la forma $\times_{i=1}^d]a_i, b_i]$, con $-\infty < a_i \leq b_i < \infty$, se tiene

$$\lambda\left(\times_{i=1}^d]a_i, b_i]\right) = \prod_{i=1}^d (b_i - a_i).$$

Esta medida se llama la **medida de Lebesgue**.

Si μ es una medida sobre el espacio medible $(\Omega, \mathcal{A})$, entonces la terna $(\Omega, \mathcal{A}, \mu)$ se llama un **espacio de medida**. En caso de que μ sea una medida de probabilidad, se llama un **espacio de probabilidad**.

Sea $(\Omega, \mathcal{A}, \mu)$ un espacio de medida. Un subconjunto $N \subseteq \Omega$ se llama **conjunto nulo** si N es medible y $\mu(N) = 0$. Además, se dice que una igualdad o desigualdad se cumple **μ -casi en todas partes** o **μ -casi seguramente**, si se satisface en el complemento de un conjunto nulo. En caso de que μ sea claro por el contexto, simplemente escribimos “casi en todas partes” o “casi seguramente”. Usualmente esto se refiere a la medida de Lebesgue.

1.2 Variables aleatorias

1.2.1 Medibilidad de funciones

Para definir variables aleatorias, primero necesitamos recordar la medibilidad de funciones.

Definición 1.2.1. Sean $(\Omega_1, \mathcal{A}_1)$ y $(\Omega_2, \mathcal{A}_2)$ dos espacios medibles. Una función $f : \Omega_1 \rightarrow \Omega_2$ se llama **medible** si

$$f^{-1}(A_2) = \{\omega \in \Omega_1 \mid f(\omega) \in A_2\} \in \mathcal{A}_1 \quad \text{para todo } A_2 \in \mathcal{A}_2.$$

Una aplicación $X : \Omega_1 \rightarrow \Omega_2$ se llama una **Ω_2 -variable aleatoria valuada** si es medible.

Observación 1.2.2. Señalamos de nuevo los paralelos con los espacios topológicos: una función $f : \Omega_1 \rightarrow \Omega_2$ entre dos espacios topológicos $(\Omega_1, \mathcal{T}_1)$ y $(\Omega_2, \mathcal{T}_2)$ se llama continua si $f^{-1}(O_2) \in \mathcal{T}_1$ para todo $O_2 \in \mathcal{T}_2$.

Sea Ω_1 un conjunto y sea $(\Omega_2, \mathcal{A}_2)$ un espacio medible. Para $X : \Omega_1 \rightarrow \Omega_2$, podemos preguntar por la sigma-álgebra más pequeña $\mathcal{A}_X$ sobre Ω_1 , tal que X sea medible como una aplicación de $(\Omega_1, \mathcal{A}_X)$ a $(\Omega_2, \mathcal{A}_2)$. Claramente, para toda sigma-álgebra $\mathcal{A}_1$ sobre Ω_1 , X es medible como aplicación de $(\Omega_1, \mathcal{A}_1)$ a $(\Omega_2, \mathcal{A}_2)$ si y solo si todo $A \in \mathcal{A}_X$ pertenece a $\mathcal{A}_1$; en otras palabras, $\mathcal{A}_X$ es una subsigma-álgebra de $\mathcal{A}_1$. Es fácil comprobar que $\mathcal{A}_X$ está dada mediante la siguiente definición.

Definición 1.2.3. Sea $X : \Omega_1 \rightarrow \Omega_2$ una variable aleatoria. Entonces

$$\mathcal{A}_X := \{X^{-1}(A_2) \mid A_2 \in \mathcal{A}_2\} \subseteq 2^{\Omega_1}$$

es la **sigma-álgebra inducida por X sobre Ω_1** .

1.2.2 Distribución y esperanza

Ahora sea $(\Omega_1, \mathcal{A}_1, \mathbb{P})$ un espacio de probabilidad, y sea $(\Omega_2, \mathcal{A}_2)$ un espacio medible. Entonces X induce naturalmente una medida sobre $(\Omega_2, \mathcal{A}_2)$ vía

$$\mathbb{P}_X[A_2] := \mathbb{P}[X^{-1}(A_2)] \quad \text{para todo } A_2 \in \mathcal{A}_2.$$

Nótese que, debido a la medibilidad de X , se cumple $X^{-1}(A_2) \in \mathcal{A}_1$, de modo que $\mathbb{P}_X$ está bien definida.

Antes de introducir la noción de densidad, recordemos dos conceptos básicos de teoría de la medida. Si Ω es un conjunto, la **medida de conteo** sobre $(\Omega, 2^\Omega)$ es la medida $\#$ definida por

$$\#(A) := \begin{cases} \text{el número de elementos de } A, & \text{si } A \text{ es finito,} \\ \infty, & \text{si } A \text{ es infinito.} \end{cases}$$

En particular, sobre un conjunto finito o numerable, una medida de probabilidad puede entenderse como una medida absolutamente continua respecto de la medida de conteo.

Sean ahora $(\Omega, \mathcal{A})$ un espacio medible y μ, ν dos medidas sobre $(\Omega, \mathcal{A})$. Decimos que μ es **absolutamente continua respecto de ν** , y escribimos $\mu \ll \nu$, si

$$\nu(A) = 0 \implies \mu(A) = 0 \quad \text{para todo } A \in \mathcal{A}.$$

Es decir, μ no asigna masa positiva a conjuntos que son nulos para ν .

El resultado fundamental que justifica la aparición de densidades es el teorema de Radon–Nikodym.

Teorema 1.2.4 (Radon–Nikodym). Sean $(\Omega, \mathcal{A})$ un espacio medible y sean μ, ν dos medidas sigma-finitas sobre $(\Omega, \mathcal{A})$. Si $\mu \ll \nu$, entonces existe una función medible $f : \Omega \rightarrow [0, \infty]$ tal que

$$\mu(A) = \int_A f \, d\nu \quad \text{para todo } A \in \mathcal{A}.$$

Además, f es única ν -casi en todas partes. Esta función se denota por

$$f = \frac{d\mu}{d\nu}$$

y se llama la **derivada de Radon–Nikodym** de μ respecto de ν .

Si además μ es finita, entonces

$$\int_{\Omega} f \, d\nu = \mu(\Omega) < \infty,$$

de modo que $f \in L^1(\nu)$. En particular, si μ es una medida de probabilidad, entonces $d\mu/d\nu \in L^1(\nu)$ y

$$\int_{\Omega} \frac{d\mu}{d\nu} \, d\nu = 1.$$

En particular, si $\mu \ll \lambda$, donde λ es la medida de Lebesgue sobre $(\mathbb{R}^d, \mathcal{B}_d)$, entonces existe una función integrable $f : \mathbb{R}^d \rightarrow [0, \infty]$ tal que

$$\mu(A) = \int_A f(x) \, dx \quad \text{para todo } A \in \mathcal{B}_d.$$

En este caso decimos que μ tiene densidad f respecto de la medida de Lebesgue. Si, en cambio, μ es una medida de probabilidad sobre un conjunto finito o numerable Ω , entonces $\mu \ll \#$, y su densidad respecto de la medida de conteo es simplemente la función de masa

$$p(\omega) = \frac{d\mu}{d\#}(\omega) = \mu(\{\omega\}).$$

Definición 1.2.5. La medida $\mathbb{P}_X$ se llama la **distribución** de X . Si $(\Omega_2, \mathcal{A}_2) = (\mathbb{R}^d, \mathcal{B}_d)$, y $\mathbb{P}_X \ll \lambda$, donde λ denota la medida de Lebesgue sobre $\mathbb{R}^d$, entonces por el teorema de Radon–Nikodym existe una función medible $f_X : \mathbb{R}^d \rightarrow [0, \infty]$ tal que

$$\mathbb{P}_X[A] = \int_A f_X(x) \, dx \quad \text{para todo } A \in \mathcal{B}_d.$$

Equivalentemente,

$$f_X = \frac{d\mathbb{P}_X}{d\lambda}.$$

Entonces f_X se llama la **densidad de Lebesgue** de X .

En probabilidad aplicada se asume con frecuencia que las distribuciones inducidas por las variables aleatorias reales o vectoriales son absolutamente continuas respecto de la medida de Lebesgue. Bajo esta hipótesis, la distribución de X puede describirse mediante su densidad f_X , y las probabilidades se calculan mediante integrales de la forma

$$\mathbb{P}[X \in A] = \mathbb{P}_X[A] = \int_A f_X(x) \, dx.$$

Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, sea $X : \Omega \rightarrow \mathbb{R}^d$ una variable aleatoria valuada en $\mathbb{R}^d$. Entonces llamamos a la integral de Lebesgue

$$\mathbb{E}[X] := \int_{\Omega} X(\omega) d\mathbb{P}(\omega) = \int_{\mathbb{R}^d} x d\mathbb{P}_X(x) \quad (\text{A.2.1})$$

la **esperanza** de X . Además, para $k \in \mathbb{N}$ decimos que X tiene **k -ésimo momento finito** si $\mathbb{E}[\|X\|^k] < \infty$. De manera similar, para una medida de probabilidad μ sobre $\mathbb{R}^d$ y $k \in \mathbb{N}$, decimos que μ tiene **k -ésimo momento finito** si

$$\int_{\mathbb{R}^d} \|x\|^k d\mu(x) < \infty.$$

Además, la matriz

$$\int_{\Omega} (X(\omega) - \mathbb{E}[X])(X(\omega) - \mathbb{E}[X])^{\top} d\mathbb{P}(\omega) \in \mathbb{R}^{d \times d}$$

es la **covarianza** de $X : \Omega \rightarrow \mathbb{R}^d$. Para $d = 1$, se llama la **varianza** de X y se denota por $\mathbb{V}[X]$.

El Teorema de arriba también explica la fórmula usual para la esperanza. Si $g : \mathbb{R}^d \rightarrow \mathbb{R}$ es medible e integrable respecto de $\mathbb{P}_X$, entonces, por definición de la medida inducida,

$$\mathbb{E}[g(X)] = \int_{\Omega} g(X(\omega)) d\mathbb{P}(\omega) = \int_{\mathbb{R}^d} g(x) d\mathbb{P}_X(x).$$

Si además $\mathbb{P}_X \ll \lambda$, con densidad $f_X = d\mathbb{P}_X/d\lambda$, entonces

$$\mathbb{E}[g(X)] = \int_{\mathbb{R}^d} g(x) f_X(x) dx.$$

En particular, tomando $g(x) = x$, cuando la integral existe, se obtiene la fórmula familiar

$$\mathbb{E}[X] = \int_{\mathbb{R}^d} x f_X(x) dx.$$

La misma idea es compatible con el teorema de cambio de variables. Si $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ es un difeomorfismo C^1 , y $Y = T(X)$, entonces la distribución de Y es la medida inducida $\mathbb{P}_Y$. Cuando X tiene densidad f_X , el teorema de cambio de variables da

$$\begin{aligned} \mathbb{P}[Y \in B] &= \mathbb{P}[X \in T^{-1}(B)] \\ &= \int_{T^{-1}(B)} f_X(x) dx \\ &= \int_B f_X(T^{-1}(y)) |\det D(T^{-1})(y)| dy. \end{aligned}$$

Por tanto, Y tiene densidad

$$f_Y(y) = f_X(T^{-1}(y)) |\det D(T^{-1})(y)|.$$

Así, cuando evaluamos esperanzas de funciones de Y , recuperamos la fórmula usual

$$\mathbb{E}[h(Y)] = \int_{\mathbb{R}^d} h(y) f_Y(y) dy.$$

Observación 1.2.6. El término distribución se usa a menudo sin especificar un espacio de probabilidad y una variable aleatoria subyacentes. En este caso, “distribución” significa indistintamente “medida de probabilidad”. Por ejemplo, μ es una distribución sobre Ω_2 afirma que μ es una medida de probabilidad sobre el espacio medible $(\Omega_2, \mathcal{A}_2)$. En este caso, siempre existe un espacio de probabilidad $(\Omega_1, \mathcal{A}_1, \mathbb{P})$ y una variable aleatoria $X : \Omega_1 \rightarrow \Omega_2$ tal que $\mathbb{P}_X = \mu$; concretamente, $(\Omega_1, \mathcal{A}_1, \mathbb{P}) = (\Omega_2, \mathcal{A}_2, \mu)$ y $X(\omega) = \omega$.

Ejemplo 1.2.7. Algunas distribuciones importantes incluyen las siguientes.

- **Distribución Bernoulli.** Una variable aleatoria $X : \Omega \rightarrow \{0, 1\}$ es Bernoulli distribuida si existe $p \in [0, 1]$ tal que $\mathbb{P}[X = 1] = p$ y $\mathbb{P}[X = 0] = 1 - p$.
- **Distribución uniforme.** Una variable aleatoria $X : \Omega \rightarrow \mathbb{R}^d$ está uniformemente distribuida sobre un conjunto medible $A \in \mathcal{B}_d$, si su densidad es igual a

$$f_X(x) = \frac{1}{|A|} \mathbf{1}_A(x),$$

donde $|A| < \infty$ es la medida de Lebesgue de A .

- **Distribución gaussiana.** Una variable aleatoria $X : \Omega \rightarrow \mathbb{R}^d$ es gaussiana distribuida con media $m \in \mathbb{R}^d$ y matriz de covarianza regular $C \in \mathbb{R}^{d \times d}$, si su densidad es igual a

$$f_X(x) = \frac{1}{(2\pi \det(C))^{d/2}} \exp\left(-\frac{1}{2}(x - m)^\top C^{-1}(x - m)\right).$$

Denotamos esta distribución por $N(m, C)$.

Finalmente, recordamos diferentes variantes de convergencia para variables aleatorias.

Definición 1.2.8. Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, sea $X_j : \Omega \rightarrow \mathbb{R}^d$, $j \in \mathbb{N}$, una sucesión de variables aleatorias, y sea $X : \Omega \rightarrow \mathbb{R}^d$ también una variable aleatoria. Se dice que la sucesión

- (i) **converge casi seguramente** a X , si

$$\mathbb{P}\left[\left\{\omega \in \Omega \mid \lim_{j \rightarrow \infty} X_j(\omega) = X(\omega)\right\}\right] = 1,$$

- (ii) **converge en probabilidad** a X , si para todo $\varepsilon > 0$:

$$\lim_{j \rightarrow \infty} \mathbb{P}\{\{\omega \in \Omega \mid \|X_j(\omega) - X(\omega)\| \geq \varepsilon\}\} = 0,$$

- (iii) **converge en distribución** a X , si para toda función continua y acotada $f : \mathbb{R}^d \rightarrow \mathbb{R}$:

$$\lim_{j \rightarrow \infty} \mathbb{E}[f \circ X_j] = \mathbb{E}[f \circ X].$$

Las nociones en la Definición 1.1.13 están ordenadas por fuerza decreciente, es decir, la convergencia casi segura implica la convergencia en probabilidad, y la convergencia en probabilidad implica la convergencia en distribución; véase por ejemplo [19, Capítulo 13]. Puesto que $\mathbb{E}[f \circ X] = \int_{\mathbb{R}^d} f(x) d\mathbb{P}_X(x)$, la noción de convergencia en distribución solo depende de la distribución $\mathbb{P}_X$ de X . Por tanto, también decimos que una sucesión de variables aleatorias converge en distribución hacia una medida μ .

1.3 Condicionales, marginales e independencia

En esta sección, nos concentramos en variables aleatorias valuadas en $\mathbb{R}^d$, aunque los conceptos siguientes pueden extenderse a espacios más generales.

1.3.1 Distribución conjunta y marginal

Sea de nuevo $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, y sean $X : \Omega \rightarrow \mathbb{R}^{d_X}$, $Y : \Omega \rightarrow \mathbb{R}^{d_Y}$ dos variables aleatorias. Entonces

$$Z := (X, Y) : \Omega \rightarrow \mathbb{R}^{d_X + d_Y}$$

también es una variable aleatoria. Su distribución $\mathbb{P}_Z$ es una medida sobre el espacio medible $(\mathbb{R}^{d_X + d_Y}, \mathcal{B}_{d_X + d_Y})$, y $\mathbb{P}_Z$ se denomina la **distribución conjunta** de X y Y . Por otro lado, $\mathbb{P}_X, \mathbb{P}_Y$ se llaman las **distribuciones marginales** de X, Y . Nótese que

$$\mathbb{P}_X[A] = \mathbb{P}_Z[A \times \mathbb{R}^{d_Y}] \quad \text{para todo } A \in \mathcal{B}_{d_X},$$

y análogamente para $\mathbb{P}_Y$. Así, las marginales $\mathbb{P}_X, \mathbb{P}_Y$ pueden construirse a partir de la distribución conjunta $\mathbb{P}_Z$. A su vez, el conocimiento de las marginales no es suficiente para construir la distribución conjunta.

1.3.2 Independencia

El concepto de independencia sirve para formalizar la situación en la que conocer una variable aleatoria no proporciona información sobre otra variable aleatoria. Primero damos la definición formal, y después discutimos el lanzamiento de un dado como un ejemplo simple.

Definición 1.3.1. Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad. Entonces dos eventos $A, B \in \mathcal{A}$ se llaman **independientes** si

$$\mathbb{P}[A \cap B] = \mathbb{P}[A]\mathbb{P}[B].$$

Dos variables aleatorias $X : \Omega \rightarrow \mathbb{R}^{d_X}$ y $Y : \Omega \rightarrow \mathbb{R}^{d_Y}$ se llaman **independientes**, si

$$A, B \text{ son independientes para todo } A \in \mathcal{A}_X, B \in \mathcal{A}_Y.$$

Dos variables aleatorias son, por tanto, independientes si y solo si todos los eventos en sus sigma-álgebras inducidas son independientes. Esto resulta ser equivalente a que la distribución conjunta $\mathbb{P}_{(X,Y)}$ sea igual a la medida producto $\mathbb{P}_X \otimes \mathbb{P}_Y$; esta última se caracteriza como la única medida sobre $\mathbb{R}^{d_X + d_Y}$ que satisface $\mu(A \times B) = \mathbb{P}_X[A]\mathbb{P}_Y[B]$ para todo $A \in \mathcal{B}_{d_X}$, $B \in \mathcal{B}_{d_Y}$.

Ejemplo 1.3.2. Sea $\Omega = \{1, \dots, 6\}$ el conjunto de resultados de lanzar un dado justo, sea $\mathcal{A} = 2^\Omega$ la sigma-álgebra, y sea $\mathbb{P}[\omega] = 1/6$ para todo $\omega \in \Omega$. Consideremos las tres variables aleatorias

$$X_1(\omega) = \begin{cases} 0 & \text{si } \omega \text{ es impar,} \\ 1 & \text{si } \omega \text{ es par,} \end{cases} \quad X_2(\omega) = \begin{cases} 0 & \text{si } \omega \leq 3, \\ 1 & \text{si } \omega \geq 4, \end{cases} \quad X_3(\omega) = \begin{cases} 0 & \text{si } \omega \in \{1, 2\}, \\ 1 & \text{si } \omega \in \{3, 4\}, \\ 2 & \text{si } \omega \in \{5, 6\}. \end{cases}$$

Estas variables aleatorias pueden interpretarse como sigue:

- X_1 indica si el lanzamiento da un número impar o par.

- X_2 indica si el lanzamiento da un número a lo sumo 3 o al menos 4.
- X_3 categoriza el lanzamiento en uno de los grupos $\{1, 2\}$, $\{3, 4\}$ o $\{5, 6\}$.

Las sigma-álgebras inducidas son

$$\begin{aligned}\mathcal{A}_{X_1} &= \{\emptyset, \Omega, \{1, 3, 5\}, \{2, 4, 6\}\}, \\ \mathcal{A}_{X_2} &= \{\emptyset, \Omega, \{1, 2, 3\}, \{4, 5, 6\}\}, \\ \mathcal{A}_{X_3} &= \{\emptyset, \Omega, \{1, 2\}, \{3, 4\}, \{5, 6\}, \{1, 2, 3, 4\}, \{1, 2, 5, 6\}, \{3, 4, 5, 6\}\}.\end{aligned}$$

Dejamos al lector verificar formalmente que X_1 y X_2 no son independientes, pero X_1 y X_3 sí lo son. Esto refleja el hecho de que, por ejemplo, saber que el resultado es impar hace más probable que el número pertenezca a $\{1, 2, 3\}$ que a $\{4, 5, 6\}$. Sin embargo, este conocimiento no proporciona información sobre las tres categorías $\{1, 2\}$, $\{3, 4\}$ y $\{5, 6\}$.

Si $X : \Omega \rightarrow \mathbb{R}$, $Y : \Omega \rightarrow \mathbb{R}$ son dos variables aleatorias independientes, entonces, debido a que $\mathbb{P}_{(X,Y)} = \mathbb{P}_X \otimes \mathbb{P}_Y$,

$$\begin{aligned}\mathbb{E}[XY] &= \int_{\Omega} X(\omega)Y(\omega) d\mathbb{P}(\omega) \\ &= \int_{\mathbb{R}^2} xy d\mathbb{P}_{(X,Y)}(x, y) \\ &= \int_{\mathbb{R}} x d\mathbb{P}_X(x) \int_{\mathbb{R}} y d\mathbb{P}_Y(y) \\ &= \mathbb{E}[X]\mathbb{E}[Y].\end{aligned}$$

Usando esta observación, es fácil ver que para una sucesión de variables aleatorias reales independientes $(X_i)_{i=1}^n$ con segundos momentos acotados, se cumple la **identidad de Bienaymé**

$$\mathbb{V}\left[\sum_{i=1}^n X_i\right] = \sum_{i=1}^n \mathbb{V}[X_i]. \quad (\text{A.3.1})$$

1.3.3 Distribuciones condicionales

Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, y sean $A, B \in \mathcal{A}$ dos eventos. En caso de que $\mathbb{P}[B] > 0$, definimos

$$\mathbb{P}[A \mid B] := \frac{\mathbb{P}[A \cap B]}{\mathbb{P}[B]}, \quad (\text{A.3.2})$$

y llamamos a $\mathbb{P}[A \mid B]$ la **probabilidad condicional** de A dado B .

Ejemplo 1.3.3. Consideremos el contexto del Ejemplo 1.3.2. Sea $A = \{\omega \in \Omega \mid X_1(\omega) = 0\}$ el evento de que el resultado del lanzamiento del dado fue un número impar, y sea $B = \{\omega \in \Omega \mid X_2(\omega) = 0\}$ el evento de que el resultado fue a lo sumo 3. Entonces $\mathbb{P}[B] = 1/2$, y $\mathbb{P}[A \cap B] = 1/3$. Por tanto

$$\mathbb{P}[A \mid B] = \frac{\mathbb{P}[A \cap B]}{\mathbb{P}[B]} = \frac{1/3}{1/2} = \frac{2}{3}.$$

Esto refleja que, dado que sabemos que el resultado es a lo sumo 3, la probabilidad de que el número sea impar, es decir, esté en $\{1, 3\}$, es mayor que la probabilidad de que el número sea par, es decir, igual a 2.

La probabilidad condicional en (A.3.2) solo está bien definida si $\mathbb{P}[B] > 0$. En la práctica, a menudo encontramos el caso en el que querríamos condicionar sobre un evento de probabilidad cero.

Ejemplo 1.3.4. Consideremos el siguiente procedimiento: primero extraemos un número aleatorio $p \in [0, 1]$ de acuerdo con una distribución uniforme sobre $[0, 1]$. Después extraemos un número aleatorio $X \in \{0, 1\}$ de acuerdo con una distribución p -Bernoulli, es decir, $\mathbb{P}[X = 1] = p$ y $\mathbb{P}[X = 0] = 1 - p$. Entonces (p, X) es una variable aleatoria conjunta que toma valores en $[0, 1] \times \{0, 1\}$. ¿Cuánto es $\mathbb{P}[X = 1 \mid p = 0.5]$ en este caso? Intuitivamente, debería ser $1/2$, pero nótese que $\mathbb{P}[p = 0.5] = 0$, de modo que (A.3.2) no es significativa aquí.

Definición 1.3.5 (distribución condicional regular). Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, y sean $X : \Omega \rightarrow \mathbb{R}^{d_X}$ y $Y : \Omega \rightarrow \mathbb{R}^{d_Y}$ dos variables aleatorias. Sea $\tau_{X|Y} : \mathcal{B}_{d_X} \times \mathbb{R}^{d_Y} \rightarrow [0, 1]$ tal que

- (i) $y \mapsto \tau_{X|Y}(A, y) : \mathbb{R}^{d_Y} \rightarrow [0, 1]$ es medible para todo $A \in \mathcal{B}_{d_X}$,
- (ii) $A \mapsto \tau_{X|Y}(A, y)$ es una medida de probabilidad sobre $(\mathbb{R}^{d_X}, \mathcal{B}_{d_X})$ para todo $y \in Y(\Omega)$,
- (iii) para todo $A \in \mathcal{B}_{d_X}$ y todo $B \in \mathcal{B}_{d_Y}$ se cumple

$$\mathbb{P}[X \in A, Y \in B] = \int_B \tau_{X|Y}(A, y) d\mathbb{P}_Y(y).$$

Entonces τ se llama una **versión regular de la distribución condicional** de X dado Y . En este caso, denotamos

$$\mathbb{P}[X \in A \mid Y = y] := \tau_{X|Y}(A, y),$$

y nos referimos a esta medida como la distribución condicional de $X \mid Y = y$.

La Definición 1.1.18 proporciona una manera matemáticamente rigurosa de asignar una distribución a una variable aleatoria condicionada a un evento que puede tener probabilidad cero, como en el Ejemplo 1.3.4. La existencia y unicidad de estas distribuciones condicionales se cumplen en el siguiente sentido; véase por ejemplo [19, Capítulo 8] o [10, Capítulo 3] para el enunciado específico dado aquí.

Teorema 1.3.6. Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, y sean $X : \Omega \rightarrow \mathbb{R}^{d_X}$, $Y : \Omega \rightarrow \mathbb{R}^{d_Y}$ dos variables aleatorias. Entonces existe una versión regular de la distribución condicional τ_1 .

Sea τ_2 otra versión regular de la distribución condicional. Entonces existe un conjunto $\mathbb{P}_Y$ -nulo $N \subseteq \mathbb{R}^{d_Y}$, tal que para todo $y \in N^c \cap Y(\Omega)$, las dos medidas de probabilidad $\tau_1(\cdot, y)$ y $\tau_2(\cdot, y)$ coinciden.

En particular, las distribuciones condicionales solo están bien definidas en un sentido $\mathbb{P}_Y$ -casi en todas partes.

Definición 1.3.7. Sea $(\Omega, \mathcal{A}, \mathbb{P})$ un espacio de probabilidad, y sean $X : \Omega \rightarrow \mathbb{R}^{d_X}$, $Y : \Omega \rightarrow \mathbb{R}^{d_Y}$, $Z : \Omega \rightarrow \mathbb{R}^{d_Z}$ tres variables aleatorias. Decimos que X y Z son **condicionalmente independientes dado Y** , si las dos distribuciones $X \mid Y = y$ y $Z \mid Y = y$ son independientes para $\mathbb{P}_Y$ -casi todo $y \in Y(\Omega)$.

1.4 Desigualdades de concentración

Sea $X_i : \Omega \rightarrow \mathbb{R}$, $i \in \mathbb{N}$, una sucesión de variables aleatorias con primeros momentos finitos. El promedio centrado sobre los primeros n términos

$$S_n := \frac{1}{n} \sum_{i=1}^n (X_i - \mathbb{E}[X_i]) \quad (\text{A.4.1})$$

es otra variable aleatoria, y por linealidad de la esperanza se cumple $\mathbb{E}[S_n] = 0$. Se dice que la sucesión satisface la **ley fuerte de los grandes números** si

$$\mathbb{P} \left[\limsup_{n \rightarrow \infty} |S_n| = 0 \right] = 1.$$

Este es, por ejemplo, el caso si existen $C < \infty$ tal que $\mathbb{V}[X_i] \leq C$ para todo $i \in \mathbb{N}$. Las desigualdades de concentración proporcionan cotas sobre la tasa de esta convergencia.

Comenzamos con la desigualdad de Markov.

Lema 1.4.1 (desigualdad de Markov). *Sea $X : \Omega \rightarrow \mathbb{R}$ una variable aleatoria, y sea $\varphi : [0, \infty) \rightarrow [0, \infty)$ monótonamente creciente. Entonces para todo $\varepsilon > 0$*

$$\mathbb{P}[|X| \geq \varepsilon] \leq \frac{\mathbb{E}[\varphi(|X|)]}{\varphi(\varepsilon)}.$$

Demostración. Tenemos

$$\mathbb{P}[|X| \geq \varepsilon] = \int_{X^{-1}([\varepsilon, \infty))} 1 \, d\mathbb{P}(\omega) \leq \int_{\Omega} \frac{\varphi(|X(\omega)|)}{\varphi(\varepsilon)} \, d\mathbb{P}(\omega) = \frac{\mathbb{E}[\varphi(|X|)]}{\varphi(\varepsilon)},$$

lo cual da la afirmación. $\square$

Aplicar la desigualdad de Markov con $\varphi(x) := x^2$ a la variable aleatoria $X - \mathbb{E}[X]$ da directamente la desigualdad de Chebyshev.

Lema 1.4.2 (desigualdad de Chebyshev). *Sea $X : \Omega \rightarrow \mathbb{R}$ una variable aleatoria con varianza finita. Entonces para todo $\varepsilon > 0$*

$$\mathbb{P}[|X - \mathbb{E}[X]| \geq \varepsilon] \leq \frac{\mathbb{V}[X]}{\varepsilon^2}.$$

De la desigualdad de Chebyshev obtenemos el siguiente resultado, que es una desigualdad de concentración bastante general para variables aleatorias con varianzas finitas.

Teorema 1.4.3. *Sean $X_1, \dots, X_n$, $n \in \mathbb{N}$, variables aleatorias reales independientes tales que para algún $\varsigma > 0$ se cumple $\mathbb{E}[|X_i - \mu|^2] \leq \varsigma^2$ para todo $i = 1, \dots, n$. Denotemos*

$$\mu := \mathbb{E} \left[\frac{1}{n} \sum_{j=1}^n X_j \right]. \quad (\text{A.4.2})$$

Entonces para todo $\varepsilon > 0$

$$\mathbb{P} \left[\left| \frac{1}{n} \sum_{j=1}^n X_j - \mu \right| \geq \varepsilon \right] \leq \frac{\varsigma^2}{\varepsilon^2 n}.$$

Demostración. Sea

$$S_n = \frac{1}{n} \sum_{j=1}^n (X_j - \mathbb{E}[X_j]) = \left(\sum_{j=1}^n X_j \right) / n - \mu.$$

Por la identidad de Bienaymé (A.3.1), se cumple

$$\mathbb{V}[S_n] = \frac{1}{n^2} \sum_{j=1}^n \mathbb{E}[|X_j - \mathbb{E}[X_j]|^2] \leq \frac{\zeta^2}{n}.$$

Puesto que $\mathbb{E}[S_n] = 0$, la desigualdad de Chebyshev aplicada a S_n da el enunciado. $\square$

Si tenemos información adicional sobre las variables aleatorias, entonces podemos derivar cotas más precisas. En el caso de variables aleatorias acotadas uniformemente, más que solamente con varianza acotada, la desigualdad de Hoeffding, que recordamos a continuación, muestra una tasa exponencial de concentración alrededor de la media.

Teorema 1.4.4 (desigualdad de Hoeffding). *Sean $a, b \in \mathbb{R}$. Sean $X_1, \dots, X_n$, $n \in \mathbb{N}$, variables aleatorias reales independientes tales que $a \leq X_i \leq b$ casi seguramente para todo $i = 1, \dots, n$, y sea μ como en (A.4.2). Entonces, para todo $\varepsilon > 0$,*

$$\mathbb{P} \left[\left| \frac{1}{n} \sum_{j=1}^n X_j - \mu \right| > \varepsilon \right] \leq 2e^{-\frac{2n\varepsilon^2}{(b-a)^2}}.$$

Una demostración puede encontrarse, por ejemplo, en [32, Sección B.4], de donde también se toma esta versión. Finalmente, recordamos el teorema del límite central, en su formulación multivariada. Decimos que $(X_j)_{j \in \mathbb{N}}$ es una **sucesión i.i.d. de variables aleatorias**, si las variables aleatorias son independientes por pares e idénticamente distribuidas. Para una demostración, véase [19, Teorema 15.58].

Teorema 1.4.5 (teorema central del límite multivariado). *Sea $(X_n)_{n \in \mathbb{N}}$ una sucesión i.i.d. de variables aleatorias valuadas en $\mathbb{R}^d$, tales que $\mathbb{E}[X_n] = 0 \in \mathbb{R}^d$ y $\mathbb{E}[X_{n,i}X_{n,j}] = C_{ij}$ para todo $i, j = 1, \dots, d$. Sea*

$$Y_n := \frac{X_1 + \dots + X_n}{\sqrt{n}} \in \mathbb{R}^d.$$

Entonces Y_n converge en distribución a $N(0, C)$ cuando $n \rightarrow \infty$.

Chapter 2

Aprendizaje estadístico

En esta sección introducimos el marco básico de la teoría del aprendizaje estadístico. La idea central es que los datos observados son solo una muestra finita de una distribución de probabilidad desconocida. Por tanto, el objetivo no es simplemente ajustar bien los datos disponibles, sino encontrar una regla de predicción que tenga buen desempeño sobre nuevos datos generados por la misma distribución.

2.1 Espacio de datos y distribución poblacional

Sea $\mathcal{X}$ el espacio de entradas y sea $\mathcal{Y}$ el espacio de salidas o etiquetas. En problemas de regresión, típicamente $\mathcal{Y} \subseteq \mathbb{R}^k$, mientras que en problemas de clasificación $\mathcal{Y}$ suele ser un conjunto finito, por ejemplo $\mathcal{Y} = \{1, \dots, K\}$. El espacio de datos es entonces

$$\mathcal{Z} := \mathcal{X} \times \mathcal{Y}.$$

Definición 2.1.1. Una **distribución de datos** es una medida de probabilidad $\mathcal{D}$ sobre el espacio medible $(\mathcal{X} \times \mathcal{Y}, \mathcal{A}_{\mathcal{X}} \otimes \mathcal{A}_{\mathcal{Y}})$. Un par aleatorio

$$(X, Y) \sim \mathcal{D}$$

representa un dato genérico de la población.

La distribución $\mathcal{D}$ es usualmente desconocida. En la práctica, solo observamos una muestra finita

$$S = \{(X_1, Y_1), \dots, (X_m, Y_m)\},$$

donde cada (X_i, Y_i) se interpreta como un dato observado. El supuesto estándar es que estos datos son independientes e idénticamente distribuidos según $\mathcal{D}$.

Definición 2.1.2. Una muestra de entrenamiento de tamaño m es una sucesión

$$S = ((X_1, Y_1), \dots, (X_m, Y_m))$$

de variables aleatorias con valores en $\mathcal{X} \times \mathcal{Y}$. Decimos que S es una muestra **i.i.d.** de $\mathcal{D}$ si

$$(X_i, Y_i) \sim \mathcal{D} \quad \text{para todo } i = 1, \dots, m,$$

y las variables aleatorias $(X_1, Y_1), \dots, (X_m, Y_m)$ son independientes.

El supuesto i.i.d. formaliza la idea de que los datos de entrenamiento y los datos futuros provienen del mismo mecanismo probabilístico. Sin este supuesto, o sin alguna hipótesis que lo reemplace, no hay razón matemática para esperar que un modelo que funciona bien sobre la muestra observada funcione bien sobre datos nuevos.

2.2 Hipótesis y clases de hipótesis

Una regla de predicción es una función que asigna una salida predicha a cada entrada.

Definición 2.2.1. Una **hipótesis** es una función medible

$$h : \mathcal{X} \rightarrow \mathcal{Y}.$$

Una **clase de hipótesis** es un conjunto $\mathcal{H}$ de hipótesis.

En aprendizaje automático, la clase $\mathcal{H}$ representa la familia de modelos entre los cuales se busca escoger uno. Por ejemplo, $\mathcal{H}$ puede ser una familia de funciones lineales, árboles de decisión, máquinas de soporte vectorial, o redes neuronales de una arquitectura fija. Si $\Phi(\cdot, w)$ es una red neuronal con parámetro w , entonces la clase de hipótesis asociada puede escribirse como

$$\mathcal{H} = \{\Phi(\cdot, w) : w \in \Theta\},$$

donde Θ es el espacio de parámetros admisibles.

2.3 Funciones de pérdida

Para evaluar la calidad de una predicción, introducimos una función de pérdida. Esta mide cuánto cuesta predecir $h(x)$ cuando la etiqueta verdadera es y .

Definición 2.3.1. Una **función de pérdida** es una función medible

$$\ell : \mathcal{Y} \times \mathcal{Y} \rightarrow [0, \infty),$$

donde $\ell(h(x), y)$ mide el error de predecir $h(x)$ cuando la salida correcta es y .

En regresión, una pérdida común es la pérdida cuadrática

$$\ell(\hat{y}, y) = \frac{1}{2} \|\hat{y} - y\|^2.$$

En clasificación, una pérdida básica es la pérdida 0-1, definida por

$$\ell(\hat{y}, y) = \mathbf{1}_{\{\hat{y} \neq y\}}.$$

Esta pérdida vale 0 cuando la predicción es correcta y 1 cuando es incorrecta.

Cuando el modelo produce puntajes o probabilidades en lugar de una etiqueta directa, se usan pérdidas sustitutas, como la pérdida logística o la entropía cruzada. Estas pérdidas suelen ser más convenientes para optimización, pues son diferenciables o casi diferenciables, a diferencia de la pérdida 0-1.

2.4 Riesgo poblacional

El verdadero objetivo del aprendizaje no es minimizar el error sobre la muestra observada, sino minimizar el error esperado sobre un dato nuevo $(X, Y) \sim \mathcal{D}$.

Definición 2.4.1. Sea $\mathcal{D}$ una distribución de datos sobre $\mathcal{X} \times \mathcal{Y}$, sea ℓ una función de pérdida, y sea $h : \mathcal{X} \rightarrow \mathcal{Y}$ una hipótesis. El **riesgo poblacional** de h se define por

$$R(h) := \mathbb{E}_{(X,Y) \sim \mathcal{D}}[\ell(h(X), Y)].$$

Equivalentemente,

$$R(h) = \int_{\mathcal{X} \times \mathcal{Y}} \ell(h(x), y) d\mathcal{D}(x, y).$$

El riesgo poblacional mide el desempeño promedio de h sobre la distribución verdadera de los datos. En términos ideales, querríamos resolver

$$\inf_{h \in \mathcal{H}} R(h).$$

Sin embargo, esto no es directamente posible porque la distribución $\mathcal{D}$ es desconocida. La teoría del aprendizaje estadístico estudia cómo aproximar este problema usando únicamente la muestra observada.

2.5 Riesgo de Bayes y error de aproximación

Si pudiéramos optimizar sobre todas las funciones medibles posibles, obtendríamos el menor riesgo alcanzable bajo la distribución $\mathcal{D}$ y la pérdida ℓ .

Definición 2.5.1. El **riesgo de Bayes** se define por

$$R^* := \inf_{h: \mathcal{X} \rightarrow \mathcal{Y}} R(h),$$

donde el ínfimo se toma sobre todas las hipótesis medibles admisibles.

Un minimizador, cuando existe, se llama un **predictor de Bayes**. En general, la clase de hipótesis $\mathcal{H}$ que usamos en la práctica puede no contener un predictor de Bayes. Por tanto, incluso el mejor elemento de $\mathcal{H}$ puede tener riesgo mayor que R^* .

Definición 2.5.2. El **error de aproximación** de una clase de hipótesis $\mathcal{H}$ se define por

$$\inf_{h \in \mathcal{H}} R(h) - R^*.$$

El error de aproximación mide la limitación intrínseca de la clase $\mathcal{H}$. Si $\mathcal{H}$ es demasiado pequeña, puede ser incapaz de representar buenas reglas de predicción. Si $\mathcal{H}$ es muy grande, el error de aproximación puede disminuir, pero puede aumentar la dificultad de seleccionar una buena hipótesis a partir de datos finitos.

2.6 Riesgo empírico

Como $\mathcal{D}$ es desconocida, reemplazamos la esperanza poblacional por un promedio sobre la muestra observada.

Definición 2.6.1. Sea

$$S = ((X_1, Y_1), \dots, (X_m, Y_m))$$

una muestra de entrenamiento. El **riesgo empírico** de una hipótesis h sobre S se define por

$$\hat{R}_S(h) := \frac{1}{m} \sum_{i=1}^m \ell(h(X_i), Y_i).$$

El riesgo empírico es una variable aleatoria, pues depende de la muestra aleatoria S . Para una hipótesis fija h , si S es i.i.d. de $\mathcal{D}$, entonces

$$\mathbb{E}[\widehat{R}_S(h)] = R(h).$$

En efecto, por linealidad de la esperanza,

$$\mathbb{E}[\widehat{R}_S(h)] = \frac{1}{m} \sum_{i=1}^m \mathbb{E}[\ell(h(X_i), Y_i)] = R(h).$$

Así, para cada h fijo, el riesgo empírico es un estimador insesgado del riesgo poblacional.

2.7 Minimización del riesgo empírico

La regla más directa para escoger una hipótesis a partir de datos consiste en minimizar el riesgo empírico.

Definición 2.7.1. Un **minimizador del riesgo empírico** sobre $\mathcal{H}$ es una hipótesis $\widehat{h}_S \in \mathcal{H}$ tal que

$$\widehat{R}_S(\widehat{h}_S) = \inf_{h \in \mathcal{H}} \widehat{R}_S(h).$$

A este principio se le llama **minimización del riesgo empírico**, o ERM por sus siglas en inglés.

En la práctica, muchas veces no encontramos un minimizador exacto del riesgo empírico, sino una hipótesis que lo minimiza aproximadamente. Por ejemplo, al entrenar una red neuronal mediante descenso de gradiente estocástico o AdamW, usualmente solo obtenemos un punto aproximado y posiblemente localmente óptimo.

Si la clase de hipótesis está parametrizada por Θ , de modo que

$$\mathcal{H} = \{h_\theta : \theta \in \Theta\},$$

entonces la minimización del riesgo empírico se escribe como un problema de optimización sobre los parámetros:

$$\inf_{\theta \in \Theta} \widehat{R}_S(h_\theta) = \inf_{\theta \in \Theta} \frac{1}{m} \sum_{i=1}^m \ell(h_\theta(X_i), Y_i).$$

Este es el puente directo entre teoría estadística y optimización: el entrenamiento de un modelo es la minimización, exacta o aproximada, de una función objetivo empírica.

2.8 Generalización

El problema central es que una hipótesis puede tener bajo riesgo empírico y, sin embargo, alto riesgo poblacional. En tal caso, el modelo ha ajustado peculiaridades de la muestra de entrenamiento que no representan la distribución subyacente. Este fenómeno se conoce como sobreajuste.

Definición 2.8.1. La **brecha de generalización** de una hipótesis h respecto de una muestra S se define por

$$R(h) - \widehat{R}_S(h).$$

También se considera con frecuencia su valor absoluto,

$$|R(h) - \widehat{R}_S(h)|.$$

Una meta fundamental de la teoría del aprendizaje estadístico es controlar esta brecha. Para una hipótesis fija h , la ley de los grandes números sugiere que

$$\widehat{R}_S(h) \rightarrow R(h) \quad \text{cuando } m \rightarrow \infty.$$

Sin embargo, en aprendizaje no escogemos h antes de ver los datos. La hipótesis $\widehat{h}_S$ depende de la muestra S . Por tanto, necesitamos controlar simultáneamente la diferencia entre riesgo empírico y riesgo poblacional para muchas hipótesis.

Definición 2.8.2. La **brecha uniforme de generalización** sobre una clase $\mathcal{H}$ se define por

$$\sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)|.$$

Si esta cantidad es pequeña, entonces el riesgo empírico aproxima uniformemente al riesgo poblacional sobre toda la clase $\mathcal{H}$. En tal caso, minimizar el riesgo empírico es una estrategia razonable para obtener una hipótesis con riesgo poblacional pequeño.

2.9 Descomposición básica del exceso de riesgo

Sea

$$h_{\mathcal{H}}^* \in \arg \min_{h \in \mathcal{H}} R(h)$$

un minimizador del riesgo poblacional dentro de $\mathcal{H}$, suponiendo que existe, y sea $\widehat{h}_S$ un minimizador del riesgo empírico. Queremos controlar el exceso de riesgo

$$R(\widehat{h}_S) - R(h_{\mathcal{H}}^*).$$

Usando que $\widehat{h}_S$ minimiza el riesgo empírico, tenemos

$$\begin{aligned} R(\widehat{h}_S) - R(h_{\mathcal{H}}^*) &= R(\widehat{h}_S) - \widehat{R}_S(\widehat{h}_S) + \widehat{R}_S(\widehat{h}_S) - \widehat{R}_S(h_{\mathcal{H}}^*) \\ &\quad + \widehat{R}_S(h_{\mathcal{H}}^*) - R(h_{\mathcal{H}}^*) \\ &\leq R(\widehat{h}_S) - \widehat{R}_S(\widehat{h}_S) + \widehat{R}_S(h_{\mathcal{H}}^*) - R(h_{\mathcal{H}}^*) \\ &\leq 2 \sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)|. \end{aligned}$$

Así obtenemos la cota fundamental

$$R(\widehat{h}_S) - R(h_{\mathcal{H}}^*) \leq 2 \sup_{h \in \mathcal{H}} |R(h) - \widehat{R}_S(h)|.$$

Esta desigualdad muestra que controlar uniformemente la brecha entre riesgo empírico y riesgo poblacional permite controlar el exceso de riesgo del minimizador empírico.

Si comparamos contra el riesgo de Bayes R^* , entonces

$$R(\widehat{h}_S) - R^* = (R(\widehat{h}_S) - R(h_{\mathcal{H}}^*)) + (R(h_{\mathcal{H}}^*) - R^*).$$

El primer término es el **error de estimación**, asociado a tener una muestra finita, mientras que el segundo es el **error de aproximación**, asociado a la limitación de la clase $\mathcal{H}$.

2.10 Sobreajuste y regularización

Una clase de hipótesis muy grande puede tener bajo riesgo empírico, pero mala generalización. Intuitivamente, si $\mathcal{H}$ es demasiado flexible, puede adaptarse al ruido de la muestra de entrenamiento.

La regularización busca restringir o penalizar la complejidad de las hipótesis. En una clase parametrizada, una forma común consiste en minimizar

$$\widehat{R}_S(h_\theta) + \lambda \Omega(\theta),$$

donde $\Omega(\theta)$ es un término de penalización y $\lambda > 0$ controla la intensidad de la regularización. Por ejemplo, la regularización cuadrática usa

$$\Omega(\theta) = \frac{1}{2} \|\theta\|^2.$$

En redes neuronales, una técnica relacionada es el decaimiento de pesos, que penaliza o reduce la magnitud de los parámetros. Para optimizadores adaptativos como Adam, la variante AdamW implementa este decaimiento de manera desacoplada respecto del gradiente adaptativo.

2.11 Clasificación y regresión

En regresión, el objetivo es predecir valores continuos. Por ejemplo, si $\mathcal{Y} = \mathbb{R}$, una hipótesis es una función

$$h : \mathcal{X} \rightarrow \mathbb{R}.$$

Con pérdida cuadrática, el riesgo es

$$R(h) = \mathbb{E}[(h(X) - Y)^2].$$

En este caso, si se permite optimizar sobre todas las funciones medibles y las esperanzas existen, el predictor óptimo está dado por la esperanza condicional

$$h^*(x) = \mathbb{E}[Y \mid X = x].$$

En clasificación, el objetivo es predecir una etiqueta discreta. Si $\mathcal{Y} = \{1, \dots, K\}$, una hipótesis es una función

$$h : \mathcal{X} \rightarrow \{1, \dots, K\}.$$

Con pérdida 0-1, el riesgo es

$$R(h) = \mathbb{P}[h(X) \neq Y].$$

El clasificador de Bayes predice una clase de máxima probabilidad condicional:

$$h^*(x) \in \arg \max_{y \in \{1, \dots, K\}} \mathbb{P}[Y = y \mid X = x].$$

2.12 Entropía cruzada y modelos probabilísticos

En muchos modelos modernos, especialmente redes neuronales para clasificación, la salida no se interpreta directamente como una etiqueta, sino como un vector de probabilidades. Para K clases, el modelo produce

$$p_\theta(x) = (p_{\theta,1}(x), \dots, p_{\theta,K}(x)) \in \Delta^{K-1},$$

donde

$$\Delta^{K-1} := \left\{ p \in [0, 1]^K \left| \sum_{k=1}^K p_k = 1 \right. \right\}$$

es el simplex de probabilidad.

Una manera usual de obtener tal vector es aplicar la función softmax a los logits $z_\theta(x) \in \mathbb{R}^K$:

$$p_{\theta,k}(x) = \frac{\exp(z_{\theta,k}(x))}{\sum_{j=1}^K \exp(z_{\theta,j}(x))} \quad \text{para } k = 1, \dots, K.$$

Definición 2.12.1. Para clasificación multiclase, la **pérdida de entropía cruzada** se define por

$$\ell(p, y) := -\log p_y,$$

donde $p \in \Delta^{K-1}$ es el vector de probabilidades predichas y $y \in \{1, \dots, K\}$ es la clase verdadera.

Si el modelo está parametrizado por θ , entonces el riesgo empírico de entropía cruzada es

$$\hat{R}_S(\theta) = -\frac{1}{m} \sum_{i=1}^m \log p_{\theta, Y_i}(X_i).$$

Minimizar esta cantidad equivale a maximizar la verosimilitud condicional de las etiquetas observadas, bajo el modelo probabilístico $p_\theta(y | x)$.

Chapter 3

Redes neuronales

En esta sección introducimos la definición matemática de una red neuronal feedforward. La idea central es construir funciones complicadas mediante la composición de transformaciones afines y no linealidades simples. En aprendizaje profundo, estas funciones se usan como familias paramétricas de modelos: los parámetros son los pesos y sesgos de la red, y el entrenamiento consiste en ajustar dichos parámetros a partir de datos.

3.1 Capas, activaciones y parámetros

Una red neuronal feedforward se construye a partir de capas. Cada capa toma un vector de entrada, aplica una transformación afín, y luego aplica una función no lineal componente a componente. La función no lineal se denomina **función de activación**.

Definición 3.1.1. Sea $d \in \mathbb{N}$. Una **función de activación** es una función

$$\sigma : \mathbb{R} \rightarrow \mathbb{R}.$$

Si $x = (x_1, \dots, x_d) \in \mathbb{R}^d$, escribimos también

$$\sigma(x) := (\sigma(x_1), \dots, \sigma(x_d)) \in \mathbb{R}^d.$$

Es decir, la activación se aplica componente a componente.

Ejemplos usuales de funciones de activación son la función sigmoide, la tangente hiperbólica y la función ReLU, dadas respectivamente por

$$\sigma(x) = \frac{1}{1 + e^{-x}}, \quad \sigma(x) = \tanh(x), \quad \sigma(x) = \max\{0, x\}.$$

La presencia de una activación no lineal es esencial: sin ella, una composición de capas afines seguiría siendo una función afín, y por tanto la red no tendría mayor poder expresivo que una sola transformación lineal-afín.

Definición 3.1.2. Sean $d_{\text{in}}, d_{\text{out}} \in \mathbb{N}$. Una **capa afín** de dimensión de entrada d_{in} y dimensión de salida d_{out} es una función

$$A : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}$$

de la forma

$$A(x) = Wx + b,$$

donde $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$ es una matriz de pesos y $b \in \mathbb{R}^{d_{\text{out}}}$ es un vector de sesgos.

Si a la capa afín le aplicamos una activación, obtenemos una capa neuronal.

Definición 3.1.3. Sean $d_{\text{in}}, d_{\text{out}} \in \mathbb{N}$, sea $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ una función de activación, sea $W \in \mathbb{R}^{d_{\text{out}} \times d_{\text{in}}}$, y sea $b \in \mathbb{R}^{d_{\text{out}}}$. La función

$$T : \mathbb{R}^{d_{\text{in}}} \rightarrow \mathbb{R}^{d_{\text{out}}}, \quad T(x) = \sigma(Wx + b),$$

se llama una **capa neuronal** de ancho d_{out} .

El número d_{out} se llama el **ancho** de la capa, pues corresponde al número de neuronas de dicha capa. En efecto, si W_i denota la i -ésima fila de W , entonces la i -ésima componente de $T(x)$ es

$$T_i(x) = \sigma(\langle W_i, x \rangle + b_i),$$

de modo que cada neurona calcula primero una combinación afín de las entradas y luego aplica la activación.

3.2 Redes neuronales feedforward

Una red neuronal feedforward se obtiene componiendo varias capas. La palabra feedforward indica que la información fluye en una sola dirección: desde la entrada hacia la salida, sin ciclos ni retroalimentación interna.

Definición 3.2.1. Sean $L \in \mathbb{N}$ y $d_0, \dots, d_{L+1} \in \mathbb{N}$. Una **red neuronal feedforward** de profundidad $L + 1$, dimensiones $(d_0, \dots, d_{L+1})$, y activación $\sigma : \mathbb{R} \rightarrow \mathbb{R}$, es una función

$$\Phi(\cdot, w) : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_{L+1}}$$

definida recursivamente por

$$\begin{aligned} x^{(0)} &:= x, \\ x^{(\ell+1)} &:= \sigma(W^{(\ell)}x^{(\ell)} + b^{(\ell)}) \quad \text{para } \ell = 0, \dots, L-1, \end{aligned}$$

y

$$\Phi(x, w) := W^{(L)}x^{(L)} + b^{(L)}.$$

Aquí $W^{(\ell)} \in \mathbb{R}^{d_{\ell+1} \times d_{\ell}}$ y $b^{(\ell)} \in \mathbb{R}^{d_{\ell+1}}$ son los pesos y sesgos de la capa ℓ , y

$$w := ((W^{(0)}, b^{(0)}), \dots, (W^{(L)}, b^{(L)}))$$

denota la colección total de parámetros de la red.

En esta definición, las capas $0, \dots, L-1$ se denominan **capas ocultas**, mientras que la última transformación

$$x^{(L)} \mapsto W^{(L)}x^{(L)} + b^{(L)}$$

se denomina la **capa de salida**. Es común no aplicar la activación σ en la última capa, especialmente en problemas de regresión. En problemas de clasificación, la salida puede posteriormente transformarse mediante una función adicional, por ejemplo softmax, para obtener un vector de probabilidades.

3.3 Preactivaciones y activaciones

Para el análisis de la red y, en particular, para la retropropagación, conviene distinguir entre preactivaciones y activaciones. Las preactivaciones son los valores antes de aplicar σ , mientras que las activaciones son los valores después de aplicar σ .

Definición 3.3.1. Sea $\Phi(\cdot, w)$ una red neuronal feedforward. Definimos las **preactivaciones** $\tilde{x}^{(\ell)}$ y las **activaciones** $x^{(\ell)}$ mediante

$$\begin{aligned} x^{(0)} &:= x, \\ \tilde{x}^{(\ell+1)} &:= W^{(\ell)} x^{(\ell)} + b^{(\ell)} \quad \text{para } \ell = 0, \dots, L, \end{aligned}$$

y

$$x^{(\ell+1)} := \sigma(\tilde{x}^{(\ell+1)}) \quad \text{para } \ell = 0, \dots, L-1.$$

La salida de la red es

$$\Phi(x, w) = \tilde{x}^{(L+1)} = W^{(L)} x^{(L)} + b^{(L)}.$$

Esta notación separa claramente la parte afín de cada capa de la parte no lineal. En particular, para $\ell = 0, \dots, L-1$, se tiene

$$x^{(\ell+1)} = \sigma(\tilde{x}^{(\ell+1)}),$$

mientras que la última capa produce directamente la salida del modelo.

3.4 Número de parámetros

El tamaño de una red puede medirse por el número total de parámetros entrenables. Para cada capa ℓ , la matriz $W^{(\ell)}$ tiene $d_{\ell+1}d_\ell$ entradas, y el vector $b^{(\ell)}$ tiene $d_{\ell+1}$ entradas. Por tanto, el número total de parámetros es

$$N_{\text{par}} = \sum_{\ell=0}^L (d_{\ell+1}d_\ell + d_{\ell+1}) = \sum_{\ell=0}^L d_{\ell+1}(d_\ell + 1).$$

Este número es importante porque controla, al menos parcialmente, la capacidad expresiva del modelo y el costo computacional del entrenamiento.

Definición 3.4.1. El **tamaño** de una red neuronal feedforward puede definirse como el número total de parámetros entrenables:

$$\text{size}(\Phi) := \sum_{\ell=0}^L d_{\ell+1}(d_\ell + 1).$$

Cuando solo se cuentan los parámetros no nulos, se define

$$\text{size}_0(\Phi) := \#\{\text{parámetros no nulos de } \Phi\}.$$

La cantidad $\text{size}_0(\Phi)$ es útil cuando se estudian redes dispersas, es decir, redes en las que muchos pesos son iguales a cero. En teoría de aproximación con redes neuronales, a menudo interesa estimar cuántos parámetros no nulos son necesarios para aproximar una función dada con cierto error.

3.5 La red como familia paramétrica de funciones

Una vez fijadas la arquitectura y la activación, una red neuronal define una familia paramétrica de funciones. La arquitectura fija las dimensiones $d_0, \dots, d_{L+1}$, mientras que el parámetro w determina una función particular dentro de esa familia.

Definición 3.5.1. Fijadas una arquitectura $(d_0, \dots, d_{L+1})$ y una activación σ , definimos la **clase de redes neuronales** asociada por

$$\mathcal{N}_\sigma(d_0, \dots, d_{L+1}) := \{\Phi(\cdot, w) : \mathbb{R}^{d_0} \rightarrow \mathbb{R}^{d_{L+1}} \mid w \text{ es una colección admisible de pesos y sesgos}\}.$$

Desde este punto de vista, entrenar una red neuronal consiste en escoger una función dentro de la clase $\mathcal{N}_\sigma(d_0, \dots, d_{L+1})$. Dado un conjunto de datos, esto se hace minimizando una función objetivo que mide qué tan bien la red ajusta los datos.

3.6 Interpretación geométrica

Cada capa transforma el espacio de representaciones. La entrada inicial $x \in \mathbb{R}^{d_0}$ se transforma primero en $x^{(1)} \in \mathbb{R}^{d_1}$, luego en $x^{(2)} \in \mathbb{R}^{d_2}$, y así sucesivamente hasta llegar a la salida $\Phi(x, w) \in \mathbb{R}^{d_{L+1}}$. Por tanto, una red neuronal puede verse como una sucesión de cambios de coordenadas no lineales:

$$\mathbb{R}^{d_0} \longrightarrow \mathbb{R}^{d_1} \longrightarrow \dots \longrightarrow \mathbb{R}^{d_L} \longrightarrow \mathbb{R}^{d_{L+1}}.$$

Las capas ocultas construyen representaciones intermedias de los datos. En problemas de clasificación, por ejemplo, se espera que estas representaciones hagan progresivamente más separables las clases. En problemas de regresión, se espera que codifiquen rasgos útiles para aproximar la relación entre entradas y salidas.

3.7 Observación sobre profundidad y expresividad

La profundidad de una red es el número de transformaciones composicionales que se aplican a la entrada. Aumentar la profundidad no solo aumenta el número de parámetros; también cambia la estructura de la familia de funciones que puede representarse. En particular, las redes profundas pueden reutilizar representaciones intermedias y construir funciones mediante composiciones sucesivas, lo cual puede ser más eficiente que intentar representar la misma función mediante una sola capa muy ancha.

Esta observación es una de las razones conceptuales detrás del aprendizaje profundo: en lugar de aprender directamente una función complicada de entrada a salida, se aprende una cadena de transformaciones más simples, cada una parametrizada por pesos, sesgos y una no linealidad.

Chapter 4

Modelos de lenguaje implementados en `nicolas-lm`

En esta sección describimos matemáticamente las familias de modelos de lenguaje implementadas en el repositorio `nicolas-lm`. El repositorio se concentra en modelos autorregresivos a nivel de caracteres. Esto significa que, dado un vocabulario finito de tokens, el objetivo es modelar la distribución condicional del siguiente token a partir de los tokens anteriores.

4.1 Modelado autorregresivo del lenguaje

Sea $\mathcal{V}$ un vocabulario finito de tamaño m . Identificamos cada token con un elemento de $\{0, \dots, m-1\}$. Una sucesión de tokens se escribe como

$$x_{0:T} = (x_0, x_1, \dots, x_T), \quad x_t \in \mathcal{V}.$$

Un modelo de lenguaje autorregresivo busca asignar una probabilidad a la secuencia factorizando su distribución mediante la regla de la cadena:

$$p_\theta(x_{0:T}) = p_\theta(x_0) \prod_{t=0}^{T-1} p_\theta(x_{t+1} \mid x_0, \dots, x_t).$$

En la práctica, el modelo recibe un bloque finito de contexto

$$(x_0, \dots, x_{T-1})$$

y produce, para cada posición t , un vector de logits

$$z_t \in \mathbb{R}^m.$$

Estos logits parametrizan una distribución categórica sobre el siguiente token mediante la función softmax:

$$p_\theta(x_{t+1} = j \mid x_{\leq t}) = \frac{\exp(z_{t,j})}{\sum_{r=0}^{m-1} \exp(z_{t,r})}.$$

Definición 4.1.1. Un **modelo de lenguaje autorregresivo** sobre un vocabulario finito $\mathcal{V}$ es una familia paramétrica de distribuciones condicionales

$$p_\theta(x_{t+1} \mid x_0, \dots, x_t),$$

donde $x_0, \dots, x_t \in \mathcal{V}$. El modelo es **causal** si la predicción en la posición t solo depende de tokens en posiciones $\leq t$, y no de tokens futuros.

El entrenamiento se formula como un problema de máxima verosimilitud condicional. Dada una sucesión de entrenamiento, para cada posición se quiere que el modelo asigne probabilidad alta al token verdadero siguiente. Equivalentemente, se minimiza la pérdida de entropía cruzada introducida en el Capítulo 2, aplicada a la distribución condicional $p_\theta(\cdot \mid c_i)$ sobre $\mathcal{V}$:

$$\mathcal{L}(\theta) = -\frac{1}{N} \sum_{i=1}^N \log p_\theta(y_i \mid c_i),$$

donde c_i denota el contexto usado en el ejemplo i , y y_i el token siguiente verdadero. En la implementación, los logits tienen forma $\mathbb{R}^{B \times T \times m}$ y los targets forma $\{0, \dots, m-1\}^{B \times T}$; la pérdida se evalúa entre los logits aplanados y los tokens verdaderos.

4.2 Modelo bigrama

El modelo más simple implementado en el repositorio es el modelo bigrama. Este modelo predice el siguiente token usando únicamente el token actual. En otras palabras, aproxima

$$p_\theta(x_{t+1} \mid x_0, \dots, x_t)$$

por una distribución de la forma

$$p_\theta(x_{t+1} \mid x_t).$$

Por tanto, el modelo ignora todo el contexto anterior excepto el último token. Esta hipótesis no es matemáticamente neutral: equivale a postular que la sucesión observada es una **cadena de Markov de orden uno** sobre $\mathcal{V}$.

Definición 4.2.1 (cadena de Markov de orden uno). Una sucesión de variables aleatorias $(X_0, X_1, X_2, \dots)$ con valores en un conjunto finito $\mathcal{V}$ se llama **cadena de Markov de orden uno** si, para todo $t \geq 0$ y todo $x_0, \dots, x_{t+1} \in \mathcal{V}$,

$$\mathbb{P}(X_{t+1} = x_{t+1} \mid X_0 = x_0, \dots, X_t = x_t) = \mathbb{P}(X_{t+1} = x_{t+1} \mid X_t = x_t).$$

La función $(i, j) \mapsto \mathbb{P}(X_{t+1} = j \mid X_t = i)$ se llama la **matriz de transición** de la cadena. Si esta función no depende de t , la cadena se dice **homogénea**.

El modelo bigrama postula que el corpus es la realización de una cadena de Markov homogénea de orden uno, y la matriz de transición es el objeto a estimar.

Definición 4.2.2 (modelo bigrama). Sea $\mathcal{V}$ un vocabulario finito con $|\mathcal{V}| = m$. Un **modelo bigrama** es una familia de distribuciones condicionales

$$p_\theta(x_{t+1} = j \mid x_t = i), \quad i, j \in \{0, \dots, m-1\}.$$

Equivalentemente, el modelo está determinado por una matriz de logits

$$A \in \mathbb{R}^{m \times m},$$

donde la fila $A_i \in \mathbb{R}^m$ contiene los logits de la distribución del siguiente token, condicionada al evento $x_t = i$. La probabilidad de transición se obtiene aplicando la función softmax fila a fila:

$$p_\theta(x_{t+1} = j \mid x_t = i) = \text{softmax}(A_i)_j = \frac{\exp(A_{ij})}{\sum_{r=0}^{m-1} \exp(A_{ir})}.$$

Observación 4.2.3 (número de parámetros). El modelo tiene m^2 parámetros reales (las entradas de A), de los cuales $m^2 - m$ son efectivamente independientes: cada fila tiene $m - 1$ grados de libertad porque la suma de probabilidades es 1, lo cual permite fijar una entrada como referencia. En el código de `nicolas-lm`, la matriz A se implementa mediante `Embedding(m,m)`: al consultar el índice i , la tabla devuelve directamente la fila A_i de logits, lo cual es operacionalmente equivalente a la multiplicación $\mathbf{1}_i^\top A$ donde $\mathbf{1}_i \in \mathbb{R}^m$ es el vector one-hot del token i .

Aunque la parametrización es vía logits, el estimador de máxima verosimilitud (MLE) admite una forma cerrada en términos de frecuencias empíricas.

Proposición 4.2.4 (MLE del bigrama). Sea $s = (c_0, c_1, \dots, c_{N-1}) \in \mathcal{V}^N$ un corpus de tamaño N . Para $i, j \in \mathcal{V}$ definamos

$$N_i = \#\{t \in \{0, \dots, N-2\} : c_t = i\},$$

$$N_{ij} = \#\{t \in \{0, \dots, N-2\} : c_t = i, c_{t+1} = j\}.$$

Entonces los logits $A \in \mathbb{R}^{m \times m}$ que maximizan la verosimilitud condicional

$$L(A; s) = \prod_{t=0}^{N-2} \text{softmax}(A_{c_t})_{c_{t+1}}$$

satisfacen, para todo i con $N_i > 0$,

$$\text{softmax}(A_i^*)_j = \frac{N_{ij}}{N_i}, \quad j = 0, \dots, m-1.$$

Es decir, la distribución condicional óptima coincide con la **frecuencia empírica** de transiciones.

Proof. La log-verosimilitud condicional se descompone fila por fila. Tomando logaritmo,

$$\log L(A; s) = \sum_{t=0}^{N-2} \log \text{softmax}(A_{c_t})_{c_{t+1}} = \sum_{i=0}^{m-1} \sum_{j=0}^{m-1} N_{ij} \log \text{softmax}(A_i)_j.$$

Para i fijo, el término $\sum_j N_{ij} \log \text{softmax}(A_i)_j$ depende únicamente de la fila A_i , así que el problema se separa en m subproblemas independientes. Para cada fila i con $N_i > 0$, maximizar

$$f(A_i) = \sum_j N_{ij} \log \frac{\exp(A_{ij})}{\sum_r \exp(A_{ir})} = \sum_j N_{ij} A_{ij} - N_i \log \sum_r \exp(A_{ir})$$

respecto de $A_i \in \mathbb{R}^m$. Derivando respecto de A_{ik} e igualando a cero,

$$\frac{\partial f}{\partial A_{ik}} = N_{ik} - N_i \cdot \text{softmax}(A_i)_k = 0,$$

de donde $\text{softmax}(A_i)_k = N_{ik}/N_i$. Esta solución es única en términos de las probabilidades (no de los logits, que están determinados solo hasta una constante por fila). $\square$

Observación 4.2.5 (conexión con la entropía empírica). La log-verosimilitud por símbolo en el óptimo es

$$\frac{1}{N-1} \log L(A^*; s) = - \sum_i \frac{N_i}{N-1} \hat{H}(\hat{p}_{i \rightarrow \cdot}),$$

donde $\hat{p}_{i \rightarrow \cdot} = (N_{ij}/N_i)_j$ es la distribución condicional empírica y $\hat{H}$ la entropía de Shannon (en nats). Por tanto, el MLE iguala la pérdida de entropía cruzada con la **entropía condicional empírica** $\hat{H}(X_{t+1} | X_t)$. Esta cantidad es siempre menor o igual a la entropía empírica de los 1-gramas, $\hat{H}(X_t) = H_1$, por la desigualdad clásica de información [10, Cap. 2]. La diferencia

$$\hat{I}(X_t; X_{t+1}) = H_1 - \hat{H}(X_{t+1} | X_t)$$

mide cuánta información sobre el siguiente carácter aporta el actual: es la **información mutua empírica** de un paso.

Modelo implementado. La clase `BigramLanguageModel` recibe una matriz de índices $\text{idx} \in \{0, \dots, m-1\}^{B \times T}$ y devuelve logits $z \in \mathbb{R}^{B \times T \times m}$. Internamente, la operación es $z_{b,t,:} = A_{x_{b,t},:}$. Si se proporcionan targets $y \in \{0, \dots, m-1\}^{B \times T}$, el modelo calcula la pérdida de entropía cruzada

$$\mathcal{L} = -\frac{1}{BT} \sum_{b,t} \log \text{softmax}(A_{x_{b,t}})_{y_{b,t}}.$$

La generación se realiza autorregresivamente: dado un contexto $(x_0, \dots, x_t)$, el modelo computa $\text{softmax}(A_{x_t})$ y elige aleatoriamente el siguiente token según esta distribución; luego concatena el resultado al contexto y repite.

El modelo bigrama es útil como línea base. Su limitación esencial es de *capacidad representacional*: aunque la matriz A podría tener m^2 parámetros distintos, la familia de distribuciones de Markov de orden uno no contiene dependencias más allá del último carácter. Cualquier estructura del lenguaje natural que requiera memoria de más de un símbolo (concordancia gramatical, dependencias léxicas, secuencias dígito-letra) está fuera del alcance de esta familia. Por construcción, los modelos siguientes amplían el contexto considerado.

4.3 Transformer causal pequeño

El segundo modelo implementado es un Transformer decoder-only pequeño. A diferencia del modelo bigrama, este modelo usa un bloque de contexto de longitud finita y permite que la predicción en una posición dependa de todos los tokens anteriores dentro del contexto.

Sea T el tamaño del bloque, usualmente llamado `block_size`. Dado

$$x_{0:T-1} = (x_0, \dots, x_{T-1}),$$

el modelo produce logits

$$z_0, \dots, z_{T-1}, \quad z_t \in \mathbb{R}^m,$$

donde z_t parametriza la distribución del token x_{t+1} . La causalidad impone que z_t dependa solamente de

$$x_0, \dots, x_t,$$

y no de $x_{t+1}, \dots, x_{T-1}$.

Definición 4.3.1. Un **Transformer causal decoder-only** es un modelo autorregresivo que transforma una sucesión de tokens en una sucesión de representaciones ocultas mediante bloques de autoatención causal y redes feedforward posición-a-posición. La máscara causal garantiza que la representación en la posición t solo use información de posiciones $s \leq t$.

4.3.1 Embeddings de token y posición

El primer paso consiste en asociar a cada token un vector en $\mathbb{R}^d$, donde d es la dimensión de embedding. Esto se hace mediante una matriz entrenable

$$E \in \mathbb{R}^{m \times d}.$$

Si $x_t = i$, entonces el embedding de token es

$$E_{x_t} \in \mathbb{R}^d.$$

Como el mecanismo de atención por sí mismo no distingue el orden de los tokens, se añade además un embedding posicional entrenable

$$P_t \in \mathbb{R}^d, \quad t = 0, \dots, T-1.$$

La representación inicial queda dada por

$$h_t^{(0)} = E_{x_t} + P_t.$$

En forma matricial,

$$H^{(0)} = E_X + P \in \mathbb{R}^{T \times d}.$$

4.3.2 Autoatención causal

La operación central del Transformer es la autoatención. Para cada posición, se construyen queries, keys y values mediante transformaciones lineales:

$$Q = HW_Q, \quad K = HW_K, \quad V = HW_V,$$

donde

$$W_Q, W_K, W_V \in \mathbb{R}^{d \times d_h}.$$

Para una cabeza de atención, los scores se calculan mediante productos punto escalados:

$$S = \frac{QK^\top}{\sqrt{d_h}}.$$

La máscara causal se define por

$$M_{ts} = \begin{cases} 1, & \text{si } s \leq t, \\ 0, & \text{si } s > t. \end{cases}$$

Así, la posición t puede atender a la posición s si y solo si $s \leq t$. Los scores prohibidos se reemplazan por $-\infty$ antes de aplicar softmax. Por tanto,

$$A_{ts} = \frac{\exp(S_{ts}) \mathbf{1}_{\{s \leq t\}}}{\sum_{r \leq t} \exp(S_{tr})}.$$

La salida de la cabeza es

$$\text{Attn}(H) = AV.$$

Definición 4.3.2. La autoatención causal escalada está dada por

$$\text{Attn}(H) = \text{softmax} \left(\frac{QK^\top}{\sqrt{d_h}} + C \right) V,$$

donde $C_{ts} = 0$ si $s \leq t$, y $C_{ts} = -\infty$ si $s > t$. Esta máscara impide que el modelo use información futura.

En multi-head attention, se ejecutan varias cabezas de atención en paralelo. Si hay r cabezas, cada cabeza produce una salida en $\mathbb{R}^{d_h}$, y se concatenan las salidas:

$$\text{MHA}(H) = \text{Concat}(\text{head}_1(H), \dots, \text{head}_r(H))W_O.$$

En el modelo implementado, se exige que

$$d = r d_h,$$

es decir, que la dimensión de embedding sea divisible por el número de cabezas.

A continuación enunciamos tres resultados que aclaran por qué la atención está bien definida desde el punto de vista matemático, por qué la división por $\sqrt{d_h}$ no es un detalle de implementación arbitrario y qué simetrías respeta o rompe la operación.

Proposición 4.3.3 (regularidad del softmax). *La función $\text{softmax} : \mathbb{R}^m \rightarrow \Delta^{m-1}$, donde $\Delta^{m-1} = \{p \in \mathbb{R}_{\geq 0}^m : \sum_j p_j = 1\}$ es el simplex de probabilidad, es de clase C^∞ . Su jacobiano en $z \in \mathbb{R}^m$ viene dado por*

$$\frac{\partial \text{softmax}(z)_j}{\partial z_k} = \text{softmax}(z)_j (\delta_{jk} - \text{softmax}(z)_k), \quad j, k = 0, \dots, m-1.$$

En particular, softmax es **equivariante por traslación constante**: para todo $c \in \mathbb{R}$,

$$\text{softmax}(z + c\mathbf{1}) = \text{softmax}(z).$$

Proof. Las exponenciales son de clase C^∞ y el denominador $\sum_r \exp(z_r)$ es estrictamente positivo en todo $\mathbb{R}^m$; luego softmax es C^∞ por composición. Calculamos

$$\begin{aligned} \frac{\partial}{\partial z_k} \frac{e^{z_j}}{\sum_r e^{z_r}} &= \frac{\delta_{jk} e^{z_j} \sum_r e^{z_r} - e^{z_j} e^{z_k}}{(\sum_r e^{z_r})^2} \\ &= \text{softmax}(z)_j \delta_{jk} - \text{softmax}(z)_j \text{softmax}(z)_k, \end{aligned}$$

que es la fórmula del enunciado. La equivariancia por traslación se sigue de $\exp(z_j + c) / \sum_r \exp(z_r + c) = e^c \exp(z_j) / (e^c \sum_r \exp(z_r))$. $\square$

La equivariancia por traslación es importante en la práctica: para evitar overflow numérico al computar exponenciales de logits grandes, se resta $\max_r z_r$ antes de exponenciar. El resultado es exactamente el mismo gracias a la proposición anterior. Esto es lo que hace internamente `torch.nn.functional.softmax`.

Proposición 4.3.4 (justificación del escalamiento por $\sqrt{d_h}$). *Supongamos que las entradas de Q y K son independientes e idénticamente distribuidas con media cero y varianza unitaria. Entonces, para cualesquiera filas $q_t \in \mathbb{R}^{d_h}$ de Q y $k_s \in \mathbb{R}^{d_h}$ de K ,*

$$\mathbb{E}[\langle q_t, k_s \rangle] = 0, \quad \mathbb{V}(\langle q_t, k_s \rangle) = d_h.$$

Por tanto, el producto interno escalado $\langle q_t, k_s \rangle / \sqrt{d_h}$ tiene varianza unitaria, independiente de la dimensión d_h .

Proof. Por linealidad, $\mathbb{E}[\langle q_t, k_s \rangle] = \sum_{i=1}^{d_h} \mathbb{E}[q_{t,i}] \mathbb{E}[k_{s,i}] = 0$ por independencia de las entradas y media nula. Para la varianza,

$$\mathbb{V}(\langle q_t, k_s \rangle) = \sum_{i=1}^{d_h} \mathbb{V}(q_{t,i} k_{s,i}) = \sum_{i=1}^{d_h} \mathbb{E}[q_{t,i}^2] \mathbb{E}[k_{s,i}^2] = d_h,$$

usando independencia y media nula en el segundo paso, y varianza unitaria en el tercero. La división por $\sqrt{d_h}$ iguala la varianza a uno. $\square$

Observación 4.3.5 (consecuencia numérica). Sin la escala $1/\sqrt{d_h}$, los logits del softmax tendrían varianza d_h , y para $d_h = 64$ (como en el modelo implementado) esto significaría que los argumentos del softmax fluctuarían en órdenes de $\pm\sqrt{64} = 8$. El softmax con argumentos así de grandes se vuelve casi una indicatriz (el máximo absorbe casi toda la masa), y su gradiente, dado por la Proposición anterior, se vuelve casi nulo. El escalamiento por $\sqrt{d_h}$ preserva un régimen donde el softmax es no-saturado y los gradientes son útiles para el entrenamiento.

Proposición 4.3.6 (equivariancia/invariancia bajo permutaciones). *Sea $\pi \in S_T$ una permutación de las T posiciones, y sea $P_\pi \in \mathbb{R}^{T \times T}$ la matriz de permutación asociada. Para la autoatención sin máscara,*

$$\text{Attn}^{\text{nomask}}(P_\pi H) = P_\pi \text{Attn}^{\text{nomask}}(H).$$

*Es decir, la atención sin máscara es **equivariante por permutación** de las posiciones. En consecuencia, sin embeddings posicionales, la salida no distingue $(x_0, x_1, \dots, x_{T-1})$ de cualquiera de sus permutaciones; solo el conjunto multiset $\{x_t\}$ importa. La adición de embeddings posicionales (sección anterior) rompe esta simetría.*

*La autoatención **con** máscara causal M_c no es equivariante: M_c depende del orden, y permutar las filas y columnas de M_c altera el patrón triangular.*

Proof. Sin máscara, $\text{Attn}^{\text{nomask}}(H) = \text{softmax}(QK^\top/\sqrt{d_h})V$ con $Q = HW_Q$ (y análogamente K, V). Si reemplazamos H por $P_\pi H$, entonces Q se transforma en $P_\pi Q$ (y análogamente K, V). Por tanto

$$Q'(K')^\top = P_\pi Q K^\top P_\pi^\top.$$

Como softmax actúa fila a fila y P_π permuta filas de manera invertible,

$$\text{softmax}(P_\pi Q K^\top P_\pi^\top / \sqrt{d_h}) = P_\pi \text{softmax}(Q K^\top / \sqrt{d_h}) P_\pi^\top.$$

Multiplicando por $V' = P_\pi V$,

$$\text{Attn}^{\text{nomask}}(P_\pi H) = P_\pi \text{softmax}(Q K^\top / \sqrt{d_h}) \underbrace{P_\pi^\top P_\pi}_{=I} V = P_\pi \text{Attn}^{\text{nomask}}(H). \quad \square$$

Observación 4.3.7 (número de parámetros por bloque). Un bloque Transformer pre-norm contiene los siguientes parámetros:

- Tres matrices $W_Q, W_K, W_V \in \mathbb{R}^{d \times d}$ (con $d = rd_h$) y la proyección de salida $W_O \in \mathbb{R}^{d \times d}$: en total $4d^2$ parámetros.
- Dos capas LayerNorm, cada una con $2d$ parámetros (ganancia y sesgo): $4d$ parámetros.
- Red feedforward $W_2 \text{GELU}(W_1 x + b_1) + b_2$ con dimensión oculta intermedia, típicamente $4d$: $8d^2 + 5d$ parámetros.

El total es $12d^2 + 9d$ parámetros por bloque. Para $d = 64$, esto da $12 \cdot 4096 + 576 = 49728$ parámetros por bloque; con $L = 2$ bloques, 99456 parámetros para el cuerpo del Transformer (sin contar embeddings ni cabeza de salida).

4.3.3 Bloque Transformer pre-norm

El modelo usa bloques Transformer de tipo pre-norm. Esto significa que la normalización se aplica antes de cada subcapa. Cada bloque tiene dos subcapas residuales: autoatención causal y red feedforward.

Definición 4.3.8. Un **bloque Transformer pre-norm** transforma una secuencia de representaciones H mediante

$$\begin{aligned} H' &= H + \text{MHA}(\text{LN}(H)), \\ H_{\text{out}} &= H' + \text{FFN}(\text{LN}(H')). \end{aligned}$$

Aquí LN denota LayerNorm, MHA denota autoatención causal multi-cabeza, y FFN denota una red feedforward aplicada independientemente en cada posición.

La red feedforward del Transformer pequeño tiene la forma

$$\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2,$$

donde típicamente la dimensión oculta es $4d$. En el código, esta subred actúa posición por posición, es decir, no mezcla información entre posiciones; la mezcla entre posiciones ocurre en la atención.

Después de aplicar varios bloques, el modelo aplica una normalización final y una capa lineal hacia el vocabulario:

$$z_t = W_{\text{out}} h_t + b_{\text{out}} \in \mathbb{R}^m.$$

Modelo implementado. El modelo `TinyTransformerLanguageModel` usa embeddings de token, embeddings posicionales aprendidos, una pila de bloques Transformer causales pre-norm, una normalización final y una cabeza lineal de lenguaje. Dado un tensor de índices

$$\text{idx} \in \{0, \dots, m-1\}^{B \times T},$$

devuelve logits

$$\text{logits} \in \mathbb{R}^{B \times T \times m}.$$

Durante la generación, si el contexto acumulado supera el tamaño máximo de bloque, el modelo solo conserva los últimos `block_size` tokens:

$$x_{\text{cond}} = x_{-T:}.$$

Luego calcula los logits de la última posición, aplica softmax y muestrea el siguiente token:

$$x_{t+1} \sim \text{Categorical}(\text{softmax}(z_t)).$$

4.4 Modelo estilo LLaMA

El tercer modelo implementado es una variante decoder-only inspirada en LLaMA. Mantiene la interfaz de entrenamiento del Transformer causal, pero cambia varios componentes arquitectónicos: usa RMSNorm en lugar de LayerNorm, embeddings posicionales rotatorios en la atención, proyecciones sin sesgo y bloques feedforward de tipo SwiGLU.

Definición 4.4.1. Un **modelo estilo LLaMA** en este contexto es un Transformer decoder-only causal que reemplaza algunos componentes del Transformer básico por elecciones arquitectónicas modernas: normalización RMSNorm, embeddings posicionales rotatorios, atención causal multi-cabeza con proyecciones sin sesgo, y redes feedforward de tipo SwiGLU.

4.4.1 RMSNorm

En el Transformer pequeño se usa LayerNorm. En el modelo estilo LLaMA se usa RMSNorm. La diferencia es que RMSNorm no centra el vector restando su media; solo reescala por su raíz cuadrática media y luego aplica un parámetro entrenable de escala.

Sea $x \in \mathbb{R}^d$. Definimos

$$\text{RMS}(x) = \left(\frac{1}{d} \sum_{i=1}^d x_i^2 \right)^{1/2}.$$

Entonces RMSNorm tiene la forma

$$\text{RMSNorm}(x) = g \odot \frac{x}{\sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon}},$$

donde $g \in \mathbb{R}^d$ es un parámetro entrenable y $\varepsilon > 0$ se introduce para estabilidad numérica.

Definición 4.4.2 (RMSNorm). La normalización **RMSNorm** de un vector $x \in \mathbb{R}^d$ se define por

$$\text{RMSNorm}(x) = g \odot x \left(\frac{1}{d} \sum_{i=1}^d x_i^2 + \varepsilon \right)^{-1/2},$$

donde $g \in \mathbb{R}^d$ es un parámetro entrenable y $\varepsilon > 0$ es una constante de estabilidad numérica. A diferencia de LayerNorm, RMSNorm no resta la media de las coordenadas.

Proposición 4.4.3 (RMSNorm como LayerNorm sin centrado). Sea $\text{LayerNorm}(x) = g \odot ((x - \mu(x))/\sigma(x)) + b$ la normalización LayerNorm clásica con $\mu(x) = \frac{1}{d} \sum_i x_i$ y $\sigma(x)^2 = \frac{1}{d} \sum_i (x_i - \mu(x))^2$, donde $g, b \in \mathbb{R}^d$ son aprendibles. Entonces RMSNorm coincide con LayerNorm en la restricción $\mu(x) = 0$ y $b = 0$, pues bajo esa restricción $\sigma(x)^2 = \frac{1}{d} \sum_i x_i^2$ y por tanto $\text{LayerNorm}(x) = g \odot x/\sigma(x)$.

En particular, RMSNorm tiene d parámetros entrenables (solo g), mientras que LayerNorm tiene $2d$ (g y b).

Observación 4.4.4 (costo computacional). LayerNorm requiere dos pasadas sobre el vector x : una para computar la media $\mu(x)$ y otra para computar la varianza $\sigma(x)^2$ usando la media. RMSNorm requiere una sola pasada (computa directamente $\sum_i x_i^2$). En arquitecturas con muchas capas y dimensiones altas, esta diferencia se acumula: Zhang and Sennrich [36] reportan reducciones del orden de 7%–64% en el tiempo de inferencia respecto de LayerNorm, sin pérdida sustancial de desempeño.

4.4.2 Embeddings posicionales rotatorios

El modelo estilo LLaMA no usa embeddings posicionales aprendidos sumados a los embeddings de token. En cambio, incorpora información posicional mediante rotary positional embeddings, o RoPE, aplicados a las queries y keys de la atención.

La idea es dividir la dimensión de cada cabeza en pares de coordenadas y aplicar a cada par una rotación que depende de la posición. Si x_{2r}, x_{2r+1} son dos coordenadas consecutivas, se define

$$\begin{pmatrix} \tilde{x}_{2r} \\ \tilde{x}_{2r+1} \end{pmatrix} = \begin{pmatrix} \cos \theta_{t,r} & -\sin \theta_{t,r} \\ \sin \theta_{t,r} & \cos \theta_{t,r} \end{pmatrix} \begin{pmatrix} x_{2r} \\ x_{2r+1} \end{pmatrix},$$

donde t es la posición del token. En el código, los ángulos se construyen usando frecuencias inversas de la forma

$$\omega_r = 10000^{-2r/d_h},$$

de modo que

$$\theta_{t,r} = t\omega_r.$$

Definición 4.4.5. Los **embeddings posicionales rotatorios** aplican una rotación dependiente de la posición a cada par de coordenadas de las queries y keys:

$$(x_{2r}, x_{2r+1}) \mapsto (x_{2r} \cos \theta_{t,r} - x_{2r+1} \sin \theta_{t,r}, x_{2r} \sin \theta_{t,r} + x_{2r+1} \cos \theta_{t,r}).$$

Por esta razón, la dimensión de cada cabeza debe ser par.

En el modelo implementado, RoPE se aplica a Q y K , pero no a V :

$$Q = \text{RoPE}(HW_Q), \quad K = \text{RoPE}(HW_K), \quad V = HW_V.$$

La propiedad central de RoPE, que la distingue de los embeddings posicionales aprendidos, es que el producto interno entre queries y keys depende solo de la diferencia de posiciones, no de las posiciones absolutas.

Teorema 4.4.6 (invariancia bajo traslación relativa). *Sea $R_p : \mathbb{R}^{d_h} \rightarrow \mathbb{R}^{d_h}$ la aplicación bloque-diagonal cuyo bloque 2×2 número r es la rotación por el ángulo $p\omega_r$, donde $\omega_r = 10000^{-2r/d_h}$ y d_h es par. Entonces, para cualesquiera $q, k \in \mathbb{R}^{d_h}$ y cualesquiera posiciones $t, s \in \mathbb{Z}$,*

$$\langle R_t q, R_s k \rangle = \langle q, R_{s-t} k \rangle.$$

En particular, el producto interno depende solo de $s - t$, no de s ni t por separado.

Proof. Las matrices R_p son ortogonales (rotación bloque a bloque), de modo que $R_p^\top = R_{-p}$. Además, las rotaciones forman un grupo conmutativo en cada par 2×2 : $R_a R_b = R_{a+b}$ (la suma de ángulos). Por tanto,

$$\langle R_t q, R_s k \rangle = q^\top R_t^\top R_s k = q^\top R_{-t} R_s k = q^\top R_{s-t} k = \langle q, R_{s-t} k \rangle. \quad \square$$

Observación 4.4.7 (interpretación). Este resultado dice que el peso de atención que la posición t le asigna a la posición s , proporcional a $\langle R_t q, R_s k \rangle$, depende solo de la *distancia* $s - t$, no de la ubicación absoluta. Esto generaliza bien a contextos más largos que los vistos durante el entrenamiento: con embeddings posicionales aprendidos, cualquier posición fuera del rango de entrenamiento es un vector no entrenado; con RoPE, la operación está bien definida para cualquier $t \in \mathbb{Z}$ y su efecto sobre los productos internos es continuo en t . En la implementación, la dimensión d_h debe ser par para que el agrupamiento por pares tenga sentido.

4.4.3 Atención causal estilo LLaMA

La atención causal del modelo estilo LLaMA sigue siendo atención escalada multi-cabeza con máscara causal:

$$S = \frac{QK^\top}{\sqrt{d_h}},$$

seguida de máscara causal y softmax:

$$A_{ts} = \frac{\exp(S_{ts}) \mathbf{1}_{\{s \leq t\}}}{\sum_{r \leq t} \exp(S_{tr})}.$$

La salida es

$$\text{Attn}(H) = AVW_O.$$

La diferencia con el Transformer pequeño es que las proyecciones W_Q, W_K, W_V, W_O se implementan sin sesgo, y que Q y K incorporan información posicional mediante RoPE.

4.4.4 SwiGLU

Otra diferencia importante aparece en la red feedforward. El Transformer pequeño usa una red feedforward con GELU. El modelo estilo LLaMA usa un bloque SwiGLU.

Dado $x \in \mathbb{R}^d$, se calculan dos proyecciones hacia una dimensión oculta:

$$a = W_{\text{gate}}x, \quad b = W_{\text{up}}x.$$

Luego se aplica la función SiLU a la rama de compuerta:

$$\text{SiLU}(a) = a \odot \sigma_{\text{sigmoid}}(a),$$

y se multiplica componente a componente por la otra rama:

$$u = \text{SiLU}(a) \odot b.$$

Finalmente, se proyecta de vuelta a la dimensión de embedding:

$$\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x).$$

Definición 4.4.8 (SwiGLU). Un bloque **SwiGLU** es una red feedforward con compuerta multiplicativa dada por

$$\text{SwiGLU}(x) = W_{\text{down}}(\text{SiLU}(W_{\text{gate}}x) \odot W_{\text{up}}x),$$

donde $\text{SiLU}(z) = z \cdot \sigma_{\log}(z)$ y $\sigma_{\log}(z) = (1 + e^{-z})^{-1}$. La multiplicación componente a componente permite que una rama module la información que pasa por la otra.

Proposición 4.4.9 (propiedades de SiLU). *La función $\text{SiLU} : \mathbb{R} \rightarrow \mathbb{R}$, definida por $\text{SiLU}(z) = z \sigma_{\log}(z)$, satisface:*

1. *Es $C^\infty(\mathbb{R})$.*
2. *Su derivada es $\text{SiLU}'(z) = \sigma_{\log}(z) + z \sigma_{\log}(z)(1 - \sigma_{\log}(z))$.*
3. *Es asintóticamente equivalente a la función rectificadora: $\text{SiLU}(z) \rightarrow 0$ cuando $z \rightarrow -\infty$ y $\text{SiLU}(z) - z \rightarrow 0$ cuando $z \rightarrow +\infty$.*

4. No es monótona: tiene un mínimo global en $z^* \approx -1.278$, con valor $\text{SiLU}(z^*) \approx -0.278$.

Proof. La propiedad (1) sigue de que $\sigma_{\log}$ es C^∞ por ser composición de funciones elementales con denominador estrictamente positivo. La derivada en (2) se obtiene por la regla del producto, usando $\sigma'_{\log}(z) = \sigma_{\log}(z)(1 - \sigma_{\log}(z))$. Los límites en (3) salen de $\sigma_{\log}(z) \rightarrow 0$ cuando $z \rightarrow -\infty$ y $\sigma_{\log}(z) \rightarrow 1$ cuando $z \rightarrow +\infty$. Para (4), igualando $\text{SiLU}'(z) = 0$ y resolviendo numéricamente se obtiene $z^* \approx -1.278$; que el extremo es un mínimo se verifica calculando $\text{SiLU}''(z^*) > 0$ [11]. $\square$

La propiedad (4), no monotonía, distingue a SiLU de ReLU y de la sigmoide clásica, y se ha conjeturado que aporta capacidad adicional a las arquitecturas que la usan.

Proposición 4.4.10 (gradiente de SwiGLU respecto de x). Sean $a = W_{\text{gate}}x$, $b = W_{\text{up}}x$ y $s = \text{SiLU}(a) \odot b$. El jacobiano de SwiGLU en x es

$$\frac{\partial \text{SwiGLU}(x)}{\partial x} = W_{\text{down}} \left(\text{diag}(\text{SiLU}'(a) \odot b) W_{\text{gate}} + \text{diag}(\text{SiLU}(a)) W_{\text{up}} \right).$$

Proof. Aplicamos la regla del producto a $s = \text{SiLU}(a) \odot b$:

$$\frac{\partial s}{\partial x} = \text{diag}(\text{SiLU}'(a) \odot b) \frac{\partial a}{\partial x} + \text{diag}(\text{SiLU}(a)) \frac{\partial b}{\partial x}.$$

Sustituyendo $\partial a / \partial x = W_{\text{gate}}$ y $\partial b / \partial x = W_{\text{up}}$, y luego componiendo con W_{down} por la izquierda, obtenemos la fórmula. $\square$

Observación 4.4.11 (SwiGLU frente a FFN-GELU). El bloque feedforward estándar del Transformer es $\text{FFN}(x) = W_2 \text{GELU}(W_1 x + b_1) + b_2$, con dimensión oculta intermedia d_h^{ffn} . El bloque SwiGLU sustituye este por una operación con *dos* proyecciones de entrada (W_{gate} y W_{up}) más una de salida (W_{down}). Si todas son del mismo tamaño $d \times d_h^{\text{ffn}}$, SwiGLU tiene $3d d_h^{\text{ffn}}$ parámetros frente a $2d d_h^{\text{ffn}}$ del FFN estándar (un 50% más). Por convención, los modelos tipo LLaMA reducen d_h^{ffn} de $4d$ a aproximadamente $\frac{2}{3} \cdot 4d$ para mantener un conteo de parámetros similar; véase Touvron et al. [34].

4.4.5 Bloque Transformer estilo LLaMA

Un bloque estilo LLaMA combina RMSNorm, atención causal con RoPE y SwiGLU mediante conexiones residuales. La estructura es pre-norm:

$$\begin{aligned} H' &= H + \text{Attn}_{\text{RoPE}}(\text{RMSNorm}(H)), \\ H_{\text{out}} &= H' + \text{SwiGLU}(\text{RMSNorm}(H')). \end{aligned}$$

Modelo implementado. El modelo `LLaMAStyleLanguageModel` es un modelo causal a nivel de caracteres que usa embeddings de token, una pila de bloques estilo LLaMA, una RMSNorm final y una cabeza lineal hacia el vocabulario. A diferencia del Transformer pequeño, no usa embeddings posicionales aprendidos; la información posicional entra mediante RoPE dentro de la atención.

Dado un tensor de entrada

$$\text{idx} \in \{0, \dots, m-1\}^{B \times T},$$

el modelo calcula

$$H^{(0)} = E_{\text{tok}}(\text{idx}),$$

luego aplica N bloques estilo LLaMA:

$$H^{(\ell+1)} = \text{Block}_\ell(H^{(\ell)}), \quad \ell = 0, \dots, N-1,$$

después una RMSNorm final:

$$\tilde{H} = \text{RMSNorm}(H^{(N)}),$$

y finalmente una proyección lineal sin sesgo hacia el vocabulario:

$$z_t = W_{\text{lm}} \tilde{h}_t \in \mathbb{R}^m.$$

4.4.6 Por qué estas modificaciones son estilo LLaMA

Las modificaciones anteriores no son meramente cambios de notación. Cada una altera de manera concreta la forma en que el modelo procesa información.

Primero, RMSNorm reemplaza LayerNorm por una normalización más simple. Si $x \in \mathbb{R}^d$, LayerNorm normaliza usando la media y la varianza empíricas de las coordenadas, mientras que RMSNorm solo normaliza por la magnitud cuadrática media:

$$\text{RMS}(x) = \left(\frac{1}{d} \sum_{i=1}^d x_i^2 \right)^{1/2}.$$

Por tanto, RMSNorm preserva la dirección del vector hasta un reescalamiento componente a componente entrenable, sin centrar las coordenadas. Esto reduce ligeramente la complejidad de la normalización y es una de las decisiones arquitectónicas características de modelos estilo LLaMA.

Segundo, RoPE introduce la posición directamente dentro del mecanismo de atención, en lugar de sumar un embedding posicional aprendido a la entrada. En un Transformer con embeddings posicionales aprendidos, la representación inicial tiene la forma

$$h_t^{(0)} = E_{x_t} + P_t.$$

En cambio, con RoPE, la información posicional aparece al transformar las queries y keys:

$$Q_t \mapsto \text{RoPE}_t(Q_t), \quad K_s \mapsto \text{RoPE}_s(K_s).$$

Así, los productos internos que determinan la atención,

$$\langle Q_t, K_s \rangle,$$

pasan a depender también de las posiciones t y s . La posición no se agrega como un vector adicional al embedding inicial, sino que modifica geométricamente las direcciones usadas para calcular similitud en atención.

Tercero, SwiGLU reemplaza la red feedforward usual por una red con compuerta multiplicativa. En una red feedforward estándar, una forma típica es

$$\text{FFN}(x) = W_2 \phi(W_1 x).$$

En cambio, SwiGLU usa dos proyecciones internas:

$$\text{SwiGLU}(x) = W_{\text{down}} (\text{SiLU}(W_{\text{gate}} x) \odot W_{\text{up}} x).$$

La primera rama actúa como una compuerta, porque decide componente a componente qué información de la segunda rama se deja pasar. Esta multiplicación componente a componente aumenta la expresividad local del bloque feedforward.

El bloque completo (ya definido en la subsección anterior) conserva la estructura residual pre-norm del Transformer, sustituyendo $\text{LayerNorm} \rightarrow \text{RMSNorm}$, $\text{Attn} \rightarrow \text{Attn}_{\text{RoPE}}$ y $\text{FFN} \rightarrow \text{SwiGLU}$. Por eso el modelo sigue siendo un Transformer decoder-only, pero con elecciones arquitectónicas más cercanas a las usadas en modelos modernos tipo LLaMA.

4.5 Comparación de los tres modelos

Los tres modelos comparten la misma interfaz general: reciben índices de tokens, producen logits sobre el vocabulario, pueden calcular entropía cruzada si se proporcionan targets, y generan texto mediante muestreo autorregresivo. Sin embargo, difieren en la forma de representar el contexto.

Resumen. En `nicolas-lm` aparecen tres familias principales de modelos:

1. El **modelo bigrama**, que modela $p(x_{t+1} \mid x_t)$ y solo usa el token actual.
2. El **Transformer causal pequeño**, que modela $p(x_{t+1} \mid x_0, \dots, x_t)$ usando embeddings posicionales aprendidos, autoatención causal multi-cabeza, LayerNorm y feedforward con GELU.
3. El **modelo estilo LLaMA**, que conserva la estructura causal decoder-only, pero usa RMSNorm, RoPE, proyecciones sin sesgo y SwiGLU.

El modelo bigrama tiene el menor costo computacional y sirve como línea base. Su contexto efectivo tiene longitud 1. El Transformer causal pequeño usa un contexto de longitud `block_size` y puede aprender dependencias entre posiciones anteriores mediante atención. El modelo estilo LLaMA conserva la capacidad contextual del Transformer, pero incorpora decisiones arquitectónicas modernas que suelen mejorar la estabilidad y eficiencia del entrenamiento.

4.6 Pérdida común de entrenamiento

Aunque las arquitecturas son distintas, el objetivo de entrenamiento es el mismo. Para cada posición, el modelo produce logits sobre el vocabulario y se minimiza la entropía cruzada respecto del siguiente token verdadero. Si $z_{b,t} \in \mathbb{R}^m$ son los logits para el elemento b del batch y la posición t , y $y_{b,t} \in \{0, \dots, m-1\}$ es el token objetivo, la pérdida promedio es

$$\mathcal{L} = \frac{1}{BT} \sum_{b=1}^B \sum_{t=1}^T \text{CE}(z_{b,t}, y_{b,t}),$$

donde $\text{CE}(z, y) = -\log \text{softmax}(z)_y$ es la entropía cruzada categórica entre los logits z y el token verdadero y , tal como se definió en el Capítulo 2.

4.7 Generación autorregresiva

Los tres modelos generan texto mediante el mismo esquema conceptual. Dado un contexto inicial $x_{0:t}$, se calcula la distribución del siguiente token:

$$p_{\theta}(\cdot \mid x_{\leq t}) = \text{softmax}(z_t).$$

Luego se muestrea

$$x_{t+1} \sim p_{\theta}(\cdot \mid x_{\leq t}),$$

y se concatena el nuevo token al contexto. Repitiendo este procedimiento se obtiene una secuencia generada:

$$x_0, x_1, \dots, x_t, x_{t+1}, x_{t+2}, \dots$$

Para los modelos con tamaño máximo de contexto, si la longitud del contexto excede `block_size`, se usa únicamente la ventana más reciente:

$$x_{t-\text{block_size}+1:t}$$

4.8 Interpretación dentro del proyecto

Dentro del proyecto, estos modelos forman una progresión natural de complejidad. El modelo bigrama permite entender el caso más simple de estimación de distribuciones condicionales entre tokens. El Transformer causal introduce el mecanismo central de los modelos modernos de lenguaje: la autoatención causal. El modelo estilo LLaMA conserva la estructura Transformer, pero reemplaza componentes básicos por variantes usadas en arquitecturas modernas.

Por tanto, el repositorio no solo implementa modelos, sino que organiza una ruta conceptual:

$$\text{bigrama} \longrightarrow \text{Transformer causal} \longrightarrow \text{Transformer estilo LLaMA}.$$

Esta progresión permite estudiar cómo aumenta la capacidad de modelar dependencias contextuales a medida que se enriquecen la arquitectura y los mecanismos internos del modelo.

Chapter 5

Entrenamiento de redes neuronales

Hasta este punto, hemos discutido la representación y aproximación de ciertas clases de funciones usando redes neuronales. El segundo pilar del aprendizaje profundo concierne la pregunta de cómo ajustar una red neuronal a datos dados, es decir, habiendo fijado una arquitectura, cómo encontrar pesos y sesgos adecuados. Esto equivale a minimizar una llamada **función objetivo**, como el riesgo empírico definido en el Capítulo 2. A lo largo de este capítulo denotamos la función objetivo por

$$F : \mathbb{R}^n \rightarrow \mathbb{R},$$

y la interpretamos como una función de todos los pesos y sesgos de la red neuronal, reunidos en un vector en $\mathbb{R}^n$. El objetivo¹ es determinar aproximadamente un **minimizador**, es decir, algún $w_* \in \mathbb{R}^n$ que satisfaga

$$F(w_*) \leq F(w) \quad \text{para todo } w \in \mathbb{R}^n.$$

Los enfoques estándar incluyen principalmente variantes del descenso de gradiente estocástico. Estos son el foco del presente capítulo, en el que discutimos ideas básicas y resultados en optimización convexa usando algoritmos basados en gradiente. En las Secciones 10.1 y 10.2 exploramos el descenso de gradiente y el descenso de gradiente estocástico, y proporcionamos tasas de convergencia para objetivos suaves y fuertemente convexos. La Sección 5.3 discute tamaños de paso adaptativos y explica los principios centrales detrás de algoritmos populares como Adam. Finalmente, la Sección 5.4 introduce el algoritmo de retropropagación, que permite la aplicación eficiente de métodos basados en gradiente al entrenamiento de redes neuronales.

5.1 Descenso de gradiente

La idea general del descenso de gradiente es comenzar con algún $w_0 \in \mathbb{R}^n$, y luego aplicar actualizaciones secuenciales moviéndose en la dirección de *descenso más pronunciado* de la función objetivo. Supongamos por el momento que $F \in C^2(\mathbb{R}^n)$, y denotemos por w_k el k -ésimo iterado. Entonces

$$F(w_k + v) = F(w_k) + v^\top \nabla F(w_k) + O(\|v\|^2) \quad \text{para } \|v\| \rightarrow 0. \quad (5.1.1)$$

Esto muestra que el cambio en F alrededor de w_k está descrito localmente por el gradiente $\nabla F(w_k)$. Para v pequeño, la contribución del término de segundo orden es despreciable, y la

¹En realidad, el objetivo es más matizado: más que simplemente minimizar la función objetivo F , queremos encontrar un parámetro w que produzca un modelo que generalice bien, es decir, un riesgo poblacional pequeño (esto se discute en el Capítulo 2). Sin embargo, en este capítulo adoptamos la perspectiva de minimizar F .

dirección v a lo largo de la cual la disminución de la función objetivo es máxima es igual al gradiente negativo $-\nabla F(w_k)$. Así, $-\nabla F(w_k)$ también se llama la dirección de descenso más pronunciado. Esto conduce a una actualización de la forma

$$w_{k+1} := w_k - h_k \nabla F(w_k), \quad (5.1.2)$$

donde $h_k > 0$ se denomina el **tamaño de paso** o **tasa de aprendizaje**. Nos referimos a este algoritmo iterativo como **descenso de gradiente**.

Por (5.1.1) y (5.1.2), se tiene, también véase [6, Sección 1.2],

$$F(w_{k+1}) = F(w_k) - h_k \|\nabla F(w_k)\|^2 + O(h_k^2), \quad (5.1.3)$$

de modo que, si $\nabla F(w_k) \neq 0$, un tamaño de paso suficientemente pequeño h_k garantiza que el algoritmo disminuya el valor de la función objetivo. En la práctica, ajustar la tasa de aprendizaje h_k puede ser un asunto sutil, pues debe lograr un balance entre los siguientes requerimientos discrepantes:

- (i) h_k debe ser suficientemente pequeño para que el término de segundo orden en (5.1.3) no domine, y la actualización (5.1.2) disminuya la función objetivo.
- (ii) h_k debe ser suficientemente grande para asegurar una disminución significativa de la función objetivo, lo cual facilita una convergencia más rápida del algoritmo.

Una tasa de aprendizaje demasiado alta puede sobrepasar el mínimo, mientras que una tasa demasiado baja resulta en una convergencia lenta. Estrategias comunes incluyen, en particular, tasas de aprendizaje constantes ($h_k = h$ para todo $k \in \mathbb{N}_0$), esquemas de tasas de aprendizaje tales como tasas decrecientes ($h_k \downarrow 0$ cuando $k \rightarrow \infty$), y métodos adaptativos. Para métodos adaptativos, el algoritmo ajusta dinámicamente h_k con base en los valores de $F(w_j)$ o $\nabla F(w_j)$ para $j \leq k$, véase la Sección 5.3 más adelante.

5.1.1 Condiciones estructurales y existencia de minimizadores

Comenzamos nuestro análisis discutiendo tres nociones clave para analizar algoritmos de descenso de gradiente, empezando con una descripción geométrica intuitiva, aunque imprecisa. Una función continuamente diferenciable $F : \mathbb{R}^n \rightarrow \mathbb{R}$ será llamada

- (i) *suave* si, en cada $w \in \mathbb{R}^n$, F está acotada superior e inferiormente por una función cuadrática que toca su gráfica en w ,
- (ii) *convexa* si, en cada $w \in \mathbb{R}^n$, F yace por encima de su tangente en w ,
- (iii) *fuertemente convexa* si, en cada $w \in \mathbb{R}^n$, F yace por encima de su tangente en w más un término cuadrático convexo.

A continuación damos las definiciones matemáticas precisas.

Definición 5.1.1. Sean $n \in \mathbb{N}$ y $L > 0$. Una función $F : \mathbb{R}^n \rightarrow \mathbb{R}$ se llama **L -suave** si $F \in C^1(\mathbb{R}^n)$ y

$$F(v) \leq F(w) + \langle \nabla F(w), v - w \rangle + \frac{L}{2} \|w - v\|^2 \quad \text{para todo } w, v \in \mathbb{R}^n, \quad (5.1.4a)$$

$$F(v) \geq F(w) + \langle \nabla F(w), v - w \rangle - \frac{L}{2} \|w - v\|^2 \quad \text{para todo } w, v \in \mathbb{R}^n. \quad (5.1.4b)$$

Por definición, las funciones L -suaves satisfacen la propiedad geométrica (i). En la literatura, la L -suavidad se define a menudo en cambio como continuidad Lipschitz del gradiente:

$$\|\nabla F(w) - \nabla F(v)\| \leq L\|w - v\| \quad \text{para todo } w, v \in \mathbb{R}^n, \quad (5.1.5)$$

lo cual es equivalente, como muestra el siguiente lema; véase, por ejemplo, [3, 26].

Lema 5.1.2. *Sea $L > 0$. Entonces $F \in C^1(\mathbb{R}^n)$ es L -suave si y solo si se cumple (5.1.5).*

Demostración. Solo mostramos que (5.1.4) implica (5.1.5). El hecho de que (5.1.5) implica (5.1.4) se deja como Ejercicio 10.17.

Paso 1. Supongamos primero que $F \in C^2(\mathbb{R}^n)$, y que (5.1.5) no se cumple. Entonces podemos encontrar $w \neq v$ con

$$\|w - v\| \sup_{\|\ell\|=1} \int_0^1 \ell^\top \nabla^2 F(v + t(w - v)) \frac{w - v}{\|w - v\|} dt = \|\nabla F(w) - \nabla F(v)\| > L\|w - v\|,$$

donde $\nabla^2 F \in \mathbb{R}^{n \times n}$ denota la Hessiana. Puesto que la Hessiana es simétrica, esto implica la existencia de $u, e \in \mathbb{R}^n$ con $\|e\| = 1$ y $|e^\top \nabla^2 F(u)e| > L$. Por tanto, o bien

$$e^\top \nabla^2 F(u)e > L \quad \text{o} \quad -e^\top \nabla^2 F(u)e > L. \quad (5.1.6)$$

Supongamos que se cumple la primera desigualdad. Para $h > 0$, por la fórmula de Taylor,

$$F(u + he) = F(u) + h\langle \nabla F(u), e \rangle + \int_0^h e^\top \nabla^2 F(u + te)e(h - t) dt.$$

La continuidad de $t \mapsto e^\top \nabla^2 F(u + te)e$ y la primera desigualdad en (5.1.6) implican que, para $h > 0$ suficientemente pequeño,

$$\begin{aligned} F(u + he) &> F(u) + h\langle \nabla F(u), e \rangle + L \int_0^h (h - t) dt \\ &= F(u) + \langle \nabla F(u), he \rangle + \frac{L}{2} \|he\|^2. \end{aligned}$$

Por tanto, (5.1.4a) no se cumple. El segundo caso en (5.1.6) conduce de manera similar a una violación de (5.1.4b).

Paso 2. Supongamos ahora que $F \in C^1$ y que (5.1.4) se cumple. Sea $\rho \in C_c^\infty(\mathbb{R}^n)$ no negativa y con soporte compacto, tal que $\int_{\mathbb{R}^n} \rho(x) dx = 1$, una llamada función de mollificación, véase por ejemplo (8.1.3). Para $\varepsilon > 0$, sea $\rho_\varepsilon(x) := \varepsilon^{-n} \rho(x/\varepsilon)$. Es un resultado estándar, por ejemplo [12, Apéndice C.5], que la convolución $F_\varepsilon := F * \rho_\varepsilon \in C^\infty(\mathbb{R}^n)$ satisface

$$\lim_{\varepsilon \rightarrow 0} \nabla F_\varepsilon(v) = \nabla F(v) \quad \text{para todo } v \in \mathbb{R}^n. \quad (5.1.7)$$

Fijemos $\varepsilon > 0$. Por (5.1.4), para todo $v, w \in \mathbb{R}^n$,

$$\begin{aligned} F_\varepsilon(v) &= \int_{\mathbb{R}^n} F(v - x) \rho_\varepsilon(x) dx \\ &\leq \int_{\mathbb{R}^n} F(w - x) \rho_\varepsilon(x) + \langle \nabla F(w - x) \rho_\varepsilon(x), v - w \rangle + \frac{L}{2} \|w - v\|^2 \rho_\varepsilon(x) dx \\ &= F_\varepsilon(w) + \langle \nabla F_\varepsilon(w), v - w \rangle + \frac{L}{2} \|w - v\|^2, \end{aligned}$$

donde usamos que $\nabla F_\varepsilon = \nabla F * \rho_\varepsilon$. Por tanto, F_ε satisface (5.1.4a), y de manera similar se muestra que F_ε satisface (5.1.4b). Por el Paso 1, $F_\varepsilon \in C^\infty$ satisface (5.1.5). Finalmente, haciendo $\varepsilon \rightarrow 0$ en (5.1.7) se implica que F también satisface (5.1.5). $\square$

Definición 5.1.3. Sea $n \in \mathbb{N}$. Una función $F : \mathbb{R}^n \rightarrow \mathbb{R}$ se llama **convexa** si y solo si

$$F(\lambda w + (1 - \lambda)v) \leq \lambda F(w) + (1 - \lambda)F(v) \quad (5.1.8)$$

para todo $w, v \in \mathbb{R}^n$, $\lambda \in (0, 1)$.

En caso de que F sea continuamente diferenciable, esto es equivalente a la propiedad geométrica (ii), como muestra el siguiente lema. La demostración se deja como Ejercicio 10.18.

Lema 5.1.4. Sea $F \in C^1(\mathbb{R}^n)$. Entonces F es convexa si y solo si

$$F(v) \geq F(w) + \langle \nabla F(w), v - w \rangle \quad \text{para todo } w, v \in \mathbb{R}^n. \quad (5.1.9)$$

El concepto de convexidad se fortalece mediante la llamada fuerte convexidad, que requiere un término cuadrático positivo adicional en el lado derecho de (5.1.9), y por tanto corresponde a la propiedad geométrica (iii) por definición.

Definición 5.1.5. Sean $n \in \mathbb{N}$ y $\mu > 0$. Una función diferenciable $F : \mathbb{R}^n \rightarrow \mathbb{R}$ se llama **μ -fuertemente convexa** si

$$F(v) \geq F(w) + \langle \nabla F(w), v - w \rangle + \frac{\mu}{2} \|v - w\|^2 \quad \text{para todo } w, v \in \mathbb{R}^n. \quad (5.1.10)$$

Una función convexa no tiene por qué estar acotada inferiormente, por ejemplo $w \mapsto w$, y por tanto no tiene por qué tener minimizadores globales. E incluso si está acotada inferiormente, no necesariamente existen minimizadores, por ejemplo $w \mapsto \exp(w)$. Sin embargo, tenemos el siguiente enunciado.

Lema 5.1.6. Sea $F : \mathbb{R}^n \rightarrow \mathbb{R}$. Si F es

- (i) convexa, entonces el conjunto de minimizadores de F es convexo y tiene cardinalidad 0, 1, o ∞ ,
- (ii) μ -fuertemente convexa, entonces F tiene exactamente un minimizador.

Demostración. Sea F convexa, y supongamos que w_* y v_* son dos minimizadores de F . Entonces toda combinación convexa $\lambda w_* + (1 - \lambda)v_*$, $\lambda \in [0, 1]$, también es un minimizador por (5.1.8). Esto muestra la primera afirmación.

Ahora sea F μ -fuertemente convexa. Entonces (5.1.10) implica que F está acotada inferiormente por una función cuadrática convexa. Por tanto $\lim_{\|w\| \rightarrow \infty} F(w) = \infty$. Luego existe una sucesión acotada $(w_j)_{j \in \mathbb{N}}$ con $F(w_j) \rightarrow \inf_{w \in \mathbb{R}^n} F(w) > -\infty$. Además, para una subsucesión obtenemos $w_{j_k} \rightarrow w_*$ cuando $k \rightarrow \infty$. Puesto que F es continua, se cumple $F(w_*) = \inf_{w \in \mathbb{R}^n} F(w)$. Como w_* es un minimizador y F es diferenciable, $\nabla F(w_*) = 0$. Por (5.1.10) tenemos $F(v) > F(w_*)$ para todo $v \neq w_*$. $\square$

5.1.2 Convergencia del descenso de gradiente

Como se anunció antes, para analizar convergencia, nos concentramos únicamente en objetivos μ -fuertemente convexos y L -suaves; remitimos a la sección bibliográfica para lecturas adicionales bajo supuestos más débiles. El siguiente teorema, que establece convergencia lineal del descenso de gradiente, es un resultado estándar; véase, por ejemplo, [26, 3, 7]. La demostración presentada aquí está tomada de [13, Teorema 3.6].

Recordemos que se dice que una sucesión $(e_k)_{k \in \mathbb{N}}$ de números reales no negativos **converge linealmente** a 0, si y solo si existen constantes $C > 0$ y $c \in [0, 1)$ tales que

$$e_k \leq Cc^k \quad \text{para todo } k \in \mathbb{N}_0.$$

La constante c también se denomina la **tasa de convergencia**. Antes de dar el enunciado, primero notamos que comparar (5.1.4a) y (5.1.10) necesariamente da $L \geq \mu$, y por tanto $\kappa := L/\mu \geq 1$. Este término, conocido como el **número de condición** de F , determina la tasa de convergencia.

Teorema 5.1.7. *Sean $n \in \mathbb{N}$ y $L \geq \mu > 0$. Sea $F : \mathbb{R}^n \rightarrow \mathbb{R}$ L -suave y μ -fuertemente convexa. Además, sea $h_k = h \in (0, 1/L]$ para todo $k \in \mathbb{N}_0$, sea $(w_k)_{k=0}^\infty \subseteq \mathbb{R}^n$ definida por (5.1.2), y sea w_* el minimizador único de F .*

Entonces, $F(w_k) \rightarrow F(w_)$ y $w_k \rightarrow w_*$ convergen linealmente cuando $k \rightarrow \infty$. Para la elección específica $h = 1/L$, se cumple para todo $k \in \mathbb{N}$*

$$\|w_k - w_*\|^2 \leq \left(1 - \frac{\mu}{L}\right)^k \|w_0 - w_*\|^2, \quad (5.1.11a)$$

$$F(w_k) - F(w_*) \leq \frac{L}{2} \left(1 - \frac{\mu}{L}\right)^k \|w_0 - w_*\|^2. \quad (5.1.11b)$$

Demostración. Basta mostrar (5.1.11a), puesto que (5.1.11b) se sigue directamente de (5.1.11a) y (5.1.4a), porque $\nabla F(w_*) = 0$. El caso $k = 0$ es trivial, así que sea $k \in \mathbb{N}$.

Expandiendo $w_k = w_{k-1} - h\nabla F(w_{k-1})$ y usando μ -fuerte convexidad (5.1.10),

$$\begin{aligned} \|w_k - w_*\|^2 &= \|w_{k-1} - w_*\|^2 - 2h\langle \nabla F(w_{k-1}), w_{k-1} - w_* \rangle + h^2 \|\nabla F(w_{k-1})\|^2 \\ &\leq (1 - \mu h) \|w_{k-1} - w_*\|^2 - 2h(F(w_{k-1}) - F(w_*)) + h^2 \|\nabla F(w_{k-1})\|^2. \end{aligned} \quad (5.1.12)$$

Para acotar la suma de los dos últimos términos, primero usamos (5.1.4a) para obtener

$$\begin{aligned} F(w_k) &\leq F(w_{k-1}) + \langle \nabla F(w_{k-1}), -h\nabla F(w_{k-1}) \rangle + \frac{L}{2} \|h\nabla F(w_{k-1})\|^2 \\ &= F(w_{k-1}) + \left(\frac{L}{2} - \frac{1}{h}\right) h^2 \|\nabla F(w_{k-1})\|^2. \end{aligned}$$

Por tanto, para $h < 2/L$,

$$h^2 \|\nabla F(w_{k-1})\|^2 \leq \frac{1}{1/h - L/2} (F(w_{k-1}) - F(w_k)) \leq \frac{1}{1/h - L/2} (F(w_{k-1}) - F(w_*)),$$

y por consiguiente

$$-2h(F(w_{k-1}) - F(w_*)) + h^2 \|\nabla F(w_{k-1})\|^2 \leq \left(-2h + \frac{1}{1/h - L/2}\right) (F(w_{k-1}) - F(w_*)).$$

Si $2h \geq 1/(1/h - L/2)$, lo cual es equivalente a $h \leq 1/L$, entonces el último término es menor o igual a cero. Por tanto, (5.1.12) implica para $h \leq 1/L$

$$\|w_k - w_*\|^2 \leq (1 - \mu h) \|w_{k-1} - w_*\|^2 \leq \cdots \leq (1 - \mu h)^k \|w_0 - w_*\|^2.$$

Esto concluye la demostración. $\square$

Observación 5.1.8 (funciones objetivo convexas). Sea $F : \mathbb{R}^n \rightarrow \mathbb{R}$ convexa y L -suave con un minimizador en w_* . Como se muestra en el Lema 5.1.6, el minimizador no tiene por qué ser único, así que no podemos esperar $w_k \rightarrow w_*$ en general. Sin embargo, los valores objetivo todavía convergen. Específicamente, bajo estos supuestos, el siguiente resultado se cumple [26, Teorema 2.1.14, Corolario 2.1.2]. Si $h_k = h \in (0, 2/L)$ para todo $k \in \mathbb{N}_0$ y $(w_k)_{k=0}^\infty \subseteq \mathbb{R}^n$ es generado por (5.1.2), entonces

$$F(w_k) - F(w_*) = O(k^{-1}) \quad \text{cuando } k \rightarrow \infty.$$

5.2 Descenso de gradiente estocástico

A continuación discutimos una variante estocástica del descenso de gradiente. La idea, que originalmente se remonta a Robbins y Monro [29], es reemplazar el gradiente $\nabla F(w_k)$ en (5.1.2) por una variable aleatoria que denotamos por G_k . Interpretamos G_k como una aproximación a $\nabla F(w_k)$. Más precisamente, a lo largo de todo el texto asumimos que, dado w_k , G_k es un estimador insesgado de $\nabla F(w_k)$, condicionalmente independiente de $w_0, \dots, w_{k-1}$, véase el Apéndice A.3.3, de modo que

$$\mathbb{E}[G_k \mid w_k] \equiv \mathbb{E}[G_k \mid w_k, \dots, w_0] = \nabla F(w_k). \quad (5.2.1)$$

Después de escoger algún valor inicial $w_0 \in \mathbb{R}^n$, la regla de actualización se convierte en

$$w_{k+1} := w_k - h_k G_k, \quad (5.2.2)$$

donde $h_k > 0$ denota de nuevo el tamaño de paso. A diferencia de la Sección 5.1, nos concentramos en tamaños de paso h_k dependientes de k .

Sea w_k generado por (5.2.2). Usando L -suavidad (5.1.5), tenemos entonces [5, Lema 4.2]

$$\begin{aligned} \mathbb{E}[F(w_{k+1}) \mid w_k] - F(w_k) &\leq \mathbb{E}[\langle \nabla F(w_k), w_{k+1} - w_k \rangle \mid w_k] + \frac{L}{2} \mathbb{E}[\|w_{k+1} - w_k\|^2 \mid w_k] \\ &= -h_k \|\nabla F(w_k)\|^2 + \frac{L}{2} \mathbb{E}[\|h_k G_k\|^2 \mid w_k]. \end{aligned}$$

Para que la función objetivo disminuya en esperanza, el primer término $h_k \|\nabla F(w_k)\|^2$ debe dominar el segundo término $\frac{L}{2} \mathbb{E}[\|h_k G_k\|^2 \mid w_k]$. Como w_k se aproxima al mínimo, tenemos $\|\nabla F(w_k)\| \rightarrow 0$, lo cual sugiere que $\mathbb{E}[\|h_k G_k\|^2 \mid w_k]$ también debería disminuir con el tiempo.

Considérese, como ilustración, el descenso de gradiente estocástico (SGD) con una *tasa de aprendizaje constante*, $h_k = h$, sobre una función objetivo cuadrática y con ruido de gradiente tal que $\mathbb{E}[\|G_k\|^2 \mid w_k]$ no tiende a cero. Las actualizaciones estocásticas en (5.2.2) causan fluctuaciones en la trayectoria de optimización. Puesto que estas fluctuaciones no disminuyen a medida que el algoritmo se aproxima al mínimo, los iterados no convergerán. En cambio, se estabilizan dentro de una vecindad del mínimo, y oscilan alrededor de él, por ejemplo [13, Teorema 9.8]. En la práctica, esto podría producir una buena aproximación a w_* . Sin embargo, para alcanzar convergencia en el límite, la varianza del vector de actualización, $-h_k G_k$, debe disminuir con el tiempo. Esto puede lograrse ya sea reduciendo la varianza de G_k , por ejemplo mediante mini-lotes más grandes, cf. Ejemplo 5.2.1, o, más comúnmente, disminuyendo el tamaño de paso h_k a medida que k progresa, por ejemplo con $h_k \sim 1/k$.

5.2.1 Motivación y tasas de aprendizaje decrecientes

El siguiente ejemplo motiva el algoritmo en el contexto estándar, por ejemplo [14, Capítulo 8] o [25, Sección 8.4].

Ejemplo 5.2.1 (minimización del riesgo empírico). Supongamos que tenemos una muestra de datos $S := (x_j, y_j)_{j=1}^m$, donde $y_j \in \mathbb{R}$ es la etiqueta correspondiente al punto de datos $x_j \in \mathbb{R}^d$. Usando la pérdida cuadrática, queremos ajustar una red neuronal $\Phi(\cdot, w) : \mathbb{R}^d \rightarrow \mathbb{R}$, que depende de parámetros, es decir, pesos y sesgos $w \in \mathbb{R}^n$, de tal manera que el riesgo empírico

$$F(w) := \frac{1}{m} \sum_{j=1}^m (\Phi(x_j, w) - y_j)^2$$

sea minimizado. Realizar un paso de descenso de gradiente requiere computar

$$\nabla F(w) = \frac{2}{m} \sum_{j=1}^m (\Phi(x_j, w) - y_j) \nabla_w \Phi(x_j, w), \quad (5.2.3)$$

y por tanto computar m gradientes de la red neuronal Φ . Para m grande, en la práctica m puede estar en los millones o incluso ser mayor, este cómputo podría ser inviable.

Para reducir el costo computacional, reemplazamos el gradiente completo (5.2.3) por el gradiente estocástico

$$G := 2(\Phi(x_j, w) - y_j) \nabla_w \Phi(x_j, w),$$

donde $j \sim \text{uniform}(1, \dots, m)$ es una variable aleatoria con distribución uniforme sobre el conjunto discreto $\{1, \dots, m\}$. Entonces

$$\mathbb{E}[G] = \frac{2}{m} \sum_{j=1}^m (\Phi(x_j, w) - y_j) \nabla_w \Phi(x_j, w) = \nabla F(w),$$

lo que significa que G es un estimador insesgado de $\nabla F(w)$. Es importante notar que computar una realización de G solamente requiere una única evaluación del gradiente de la red neuronal.

Más generalmente, se pueden escoger **mini-lotes**² de tamaño m_b , donde $m_b \ll m$, y sea

$$G = \frac{2}{m_b} \sum_{j \in J} (\Phi(x_j, w) - y_j) \nabla_w \Phi(x_j, w),$$

donde J es un subconjunto aleatorio de $\{1, \dots, m\}$ de cardinalidad m_b . Un tamaño de mini-lote grande reduce la varianza de G , haciendo que sea un estimador más robusto del gradiente verdadero, pero incrementa el costo computacional.

Una variante común relacionada es la siguiente. Sea $m_b k = m$ para $m_b, k, m \in \mathbb{N}$, es decir, el número de puntos de datos m es un múltiplo k -ésimo del tamaño de mini-lote m_b . En cada **época**, primero se determina una partición aleatoria $\dot{\bigcup}_{i=1}^k J_i = \{1, \dots, m\}$, con $|J_i| = m_b$, para cada i . Entonces, para cada $i = 1, \dots, k$, los pesos se actualizan con la estimación del gradiente

$$\frac{2}{m_b} \sum_{j \in J_i} (\Phi(x_j, w) - y_j) \nabla_w \Phi(x_j, w).$$

Por tanto, en una época, correspondiente a k actualizaciones de los pesos, el algoritmo recorre todo el conjunto de datos, y cada punto de datos se usa exactamente una vez.

²En contraste con usar el lote completo, que corresponde al descenso de gradiente estándar.

5.2.2 Convergencia del descenso de gradiente estocástico

Como w_k en (5.2.2) es una variable aleatoria por construcción, un enunciado de convergencia solo puede ser estocástico, por ejemplo, en esperanza o con alta probabilidad. El siguiente teorema, que se basa en [15, Teorema 3.2] y [5, Teorema 4.7], se concentra en el primer caso. El resultado se enuncia bajo el supuesto (5.2.6), que acota los segundos momentos de los gradientes estocásticos G_k , y asegura que crecen a lo sumo linealmente con $\|\nabla F(w_k)\|^2$. Además, el análisis se apoya en los siguientes tamaños de paso decrecientes

$$h_k := \min \left(\frac{\mu}{L^2\gamma}, \frac{1}{\mu} \frac{(k+1)^2 - k^2}{(k+1)^2} \right) \quad \text{para todo } k \in \mathbb{N}_0, \quad (5.2.4)$$

de [15]. Nótese que $h_k = O(k^{-1})$ cuando $k \rightarrow \infty$, pues

$$\frac{(k+1)^2 - k^2}{(k+1)^2} = \frac{2k+1}{(k+1)^2} = \frac{2}{k+1} + O(k^{-2}). \quad (5.2.5)$$

Esta disminución de la tasa de aprendizaje nos permitirá establecer una tasa de convergencia. Sin embargo, en la práctica, podrían preferirse métodos menos agresivos de decaimiento o heurísticos que disminuyan la tasa de aprendizaje con base en el comportamiento de convergencia observado.

Teorema 5.2.2. *Sean $n \in \mathbb{N}$ y $L, \mu, \gamma > 0$. Sea $F : \mathbb{R}^n \rightarrow \mathbb{R}$ L -suave y μ -fuertemente convexa. Fijemos $w_0 \in \mathbb{R}^n$, sea $(h_k)_{k=0}^\infty$ como en (5.2.4), y sean $(G_k)_{k=0}^\infty$, $(w_k)_{k=1}^\infty$ sucesiones de variables aleatorias como en (5.2.1) y (5.2.2). Supongamos que, para algún $\gamma > 0$ fijo, y todo $k \in \mathbb{N}_0$,*

$$\mathbb{E}[\|G_k\|^2 \mid w_k] \leq \gamma(1 + \|\nabla F(w_k)\|^2). \quad (5.2.6)$$

Entonces existe una constante $C = C(w_0, \gamma, \mu, L)$ tal que para todo $k \in \mathbb{N}$

$$\begin{aligned} \mathbb{E}[\|w_k - w_*\|^2] &\leq \frac{C}{k}, \\ \mathbb{E}[F(w_k)] - F(w_*) &\leq \frac{C}{k}. \end{aligned}$$

Demostración. Usando (5.2.1) y (5.2.6), se cumple para $k \geq 1$

$$\begin{aligned} \mathbb{E}[\|w_k - w_*\|^2 \mid w_{k-1}] &= \|w_{k-1} - w_*\|^2 - 2h_{k-1}\mathbb{E}[\langle G_{k-1}, w_{k-1} - w_* \rangle \mid w_{k-1}] \\ &\quad + h_{k-1}^2\mathbb{E}[\|G_{k-1}\|^2 \mid w_{k-1}] \\ &\leq \|w_{k-1} - w_*\|^2 - 2h_{k-1}\langle \nabla F(w_{k-1}), w_{k-1} - w_* \rangle \\ &\quad + h_{k-1}^2\gamma(1 + \|\nabla F(w_{k-1})\|^2). \end{aligned}$$

Por μ -fuerte convexidad (5.1.10),

$$-2h_{k-1}\langle \nabla F(w_{k-1}), w_{k-1} - w_* \rangle \leq -\mu h_{k-1}\|w_{k-1} - w_*\|^2 - 2h_{k-1}(F(w_{k-1}) - F(w_*)).$$

Además, L -suavidad, μ -fuerte convexidad y $\nabla F(w_*) = 0$ implican

$$\|\nabla F(w_{k-1})\|^2 \leq L^2\|w_{k-1} - w_*\|^2 \leq \frac{2L^2}{\mu}(F(w_{k-1}) - F(w_*)).$$

Combinando las estimaciones anteriores obtenemos

$$\begin{aligned} \mathbb{E}[\|w_k - w_*\|^2 \mid w_{k-1}] &\leq (1 - \mu h_{k-1})\|w_{k-1} - w_*\|^2 + h_{k-1}^2\gamma \\ &\quad + 2h_{k-1} \left(\frac{L^2\gamma}{\mu} h_{k-1} - 1 \right) (F(w_{k-1}) - F(w_*)). \end{aligned}$$

La elección de $h_{k-1} \leq \mu/(L^2\gamma)$ da además

$$\mathbb{E}[\|w_k - w_*\|^2 \mid w_{k-1}] \leq (1 - \mu h_{k-1})\|w_{k-1} - w_*\|^2 + h_{k-1}^2 \gamma.$$

Nótese que ambos lados son todavía variables aleatorias. Tomando la esperanza total, y usando la **propiedad de torre**, por ejemplo [19, Satze 8.1.4], obtenemos

$$\mathbb{E}[\|w_k - w_*\|^2] \leq (1 - \mu h_{k-1})\mathbb{E}[\|w_{k-1} - w_*\|^2] + h_{k-1}^2 \gamma.$$

Con $e_0 := \|w_0 - w_*\|^2$ y $e_k := \mathbb{E}[\|w_k - w_*\|^2]$ para $k \geq 1$, hemos encontrado

$$\begin{aligned} e_k &\leq (1 - \mu h_{k-1})e_{k-1} + h_{k-1}^2 \gamma \\ &\leq (1 - \mu h_{k-1})((1 - \mu h_{k-2})e_{k-2} + h_{k-2}^2 \gamma) + h_{k-1}^2 \gamma \\ &\leq \dots \leq e_0 \prod_{j=0}^{k-1} (1 - \mu h_j) + \gamma \sum_{j=0}^{k-1} h_j^2 \prod_{i=j+1}^{k-1} (1 - \mu h_i). \end{aligned}$$

Nótese que existe $k_0 \in \mathbb{N}$ tal que por (5.2.4) y (5.2.5)

$$h_i = \frac{1}{\mu} \frac{(i+1)^2 - i^2}{(i+1)^2} \quad \text{para todo } i \geq k_0.$$

Por tanto existe $\tilde{C}$, que depende de γ, μ, L , pero es independiente de k , tal que

$$\prod_{i=j}^{k-1} (1 - \mu h_i) \leq \tilde{C} \prod_{i=j}^{k-1} \frac{i^2}{(i+1)^2} = \tilde{C} \frac{j^2}{k^2} \quad \text{para todo } 1 \leq j \leq k,$$

y además

$$\prod_{i=0}^{k-1} (1 - \mu h_i) \leq \frac{\tilde{C}}{k^2}.$$

Así,

$$\begin{aligned} \mathbb{E}[\|w_k - w_*\|^2] = e_k &\leq e_0 \frac{\tilde{C}}{k^2} + \tilde{C} \gamma \sum_{j=0}^{k-1} \frac{1}{\mu^2} \left(\frac{(j+1)^2 - j^2}{(j+1)^2} \right)^2 \frac{(j+1)^2}{k^2} \\ &\leq \frac{\tilde{C}}{k^2} e_0 + \frac{\tilde{C} \gamma}{\mu^2} \frac{1}{k^2} \sum_{j=0}^{k-1} \underbrace{\left(\frac{2j+1}{j+1} \right)^2}_{\leq 4} \\ &\leq \frac{\tilde{C}}{k^2} e_0 + \frac{\tilde{C} \gamma}{\mu^2} \frac{4k}{k^2} \leq \frac{C}{k}, \end{aligned}$$

para alguna $C = C(w_0, \gamma, \mu, L)$.

Finalmente, usando L -suavidad,

$$F(w_k) - F(w_*) \leq \langle \nabla F(w_*), w_k - w_* \rangle + \frac{L}{2} \|w_k - w_*\|^2 = \frac{L}{2} \|w_k - w_*\|^2,$$

y tomando esperanza se concluye la demostración. $\square$

La elección específica de h_k en (5.2.4) simplifica los cálculos en la demostración, pero no es necesaria para que se cumpla la convergencia asintótica. Clásicamente, la convergencia de SGD puede mostrarse bajo supuestos similares a los anteriores para tamaños de paso positivos que satisfacen $\sum_{k \in \mathbb{N}} h_k = \infty$ y $\sum_{k \in \mathbb{N}} h_k^2 < \infty$; véase por ejemplo [5, Sección 4], [7, Capítulo 4], o [29] para la referencia original.

5.3 Tasas de aprendizaje adaptativas y por coordenadas

En años recientes, se ha propuesto y estudiado una multitud de métodos de primer orden, es decir, de descenso de gradiente, para el entrenamiento de redes neuronales. Muchos de ellos incorporan algunas o todas las siguientes estrategias clave: gradientes estocásticos y tamaños de paso adaptativos. El concepto de gradientes estocásticos ya ha sido cubierto en la Sección 5.2, y ahora nos concentramos en tasas de aprendizaje adaptativas. Específicamente, siguiendo los artículos originales [9, 18, 33, 17] y en particular los resúmenes [14, 30], [1, Capítulo 11], [14, Capítulo 11], y [25, Sección 8.4], explicamos las ideas principales detrás de AdaGrad, RMSProp y Adam. Las referencias anteriores proporcionan una visión general intuitiva que cubre varias variantes adicionales que se omiten aquí. Además, en la práctica, también se usan varias otras técnicas y heurísticas, como normalización por lotes, recorte de gradiente, regularización y abandono, inicializaciones de pesos específicas, etc. No las discutimos aquí, y remitimos por ejemplo a [4, 14, 21, 25].

5.3.1 Escalamiento por coordenadas

El descenso de gradiente plano puede ser ineficiente para funciones objetivo mal condicionadas, en el sentido de que el número de condición $\kappa = L/\mu$ controla la tasa de convergencia (cf. Teorema 5.1.7). Este problema puede ser particularmente pronunciado en problemas de optimización de alta dimensión, como cuando se entrenan redes neuronales, donde ciertos parámetros influyen la salida de la red mucho más que otros. Como resultado, una sola tasa de aprendizaje puede ser subóptima; las direcciones en el espacio de parámetros con gradientes pequeños se actualizan demasiado lentamente, mientras que en direcciones con gradientes grandes el algoritmo podría sobrepasarse. Para abordar esto, un enfoque consiste en preconditionar el gradiente multiplicándolo por una matriz que tenga en cuenta la geometría del espacio de parámetros, por ejemplo [2, 23]. Una alternativa más simple y computacionalmente eficiente consiste en escalar cada componente del gradiente individualmente, de acuerdo con una cantidad diagonal de preconditionamiento. Esto permite diferentes tasas de aprendizaje para diferentes coordenadas y puede ayudar a mitigar el mal condicionamiento, pero únicamente si este mal condicionamiento está alineado con los ejes coordenados. La pregunta clave es cómo escoger las tasas de aprendizaje. La idea principal, propuesta primero en [9], consiste en escalar cada componente de manera inversamente proporcional a la magnitud de gradientes pasados. En palabras de los autores de [9]: “Nuestros procedimientos dan actualizaciones frecuentes acompañadas de tasas de aprendizaje muy bajas y características poco frecuentes tasas de aprendizaje altas.”

Después de inicializar $u_0 = 0 \in \mathbb{R}^n$, $s_0 = 0 \in \mathbb{R}^n$, y $w_0 \in \mathbb{R}^n$, todos los métodos discutidos abajo son casos especiales de

$$u_{k+1} = \beta_1 u_k + \beta_2 \nabla F(w_k), \quad (5.3.1a)$$

$$s_{k+1} = \gamma_1 s_k + \gamma_2 \nabla F(w_k) \odot \nabla F(w_k), \quad (5.3.1b)$$

$$w_{k+1} = w_k - \alpha_k u_{k+1} \oslash \sqrt{s_{k+1} + \varepsilon}, \quad (5.3.1c)$$

para $k \in \mathbb{N}_0$, y ciertos hiperparámetros $\alpha_k, \beta_1, \beta_2, \gamma_1, \gamma_2$ y ε . Aquí $\odot$ y $\oslash$ denotan la multiplicación y división componente a componente, respectivamente, y $\sqrt{s_{k+1} + \varepsilon}$ se entiende componente a componente con $\varepsilon > 0$ pequeño, para evitar división por cero en (5.3.1c). Usualmente, los métodos de tipo momentum se obtienen aplicando mini-lotes, véase la Sección 5.2. Por simplicidad los presentamos con gradientes completos.

Ejemplo 5.3.1. Consideremos una función objetivo $F : \mathbb{R}^n \rightarrow \mathbb{R}$, y su versión reescalada

$$F_\xi(w) := F(w \odot \xi), \quad \text{con gradiente } \nabla F_\xi(w) = \xi \odot \nabla F(w \odot \xi),$$

para algún $\xi \in (0, \infty)^n$. El descenso de gradiente (5.1.2) aplicado a F_ξ realiza la actualización

$$w_{k+1} = w_k - h_k \xi \odot \nabla F(w \odot \xi).$$

Poniendo $\varepsilon = 0$, (5.3.1), por otro lado, realiza la actualización

$$w_{k+1} = w_k - \alpha_k \left(\beta_2 \sum_{j=0}^k \gamma_1^j \nabla F(w_{k-j} \odot \xi) \odot \xi \right) \oslash \sqrt{\gamma_2 \sum_{j=0}^k \gamma_1^j \nabla F(w_{k-j} \odot \xi) \odot \nabla F(w_{k-j} \odot \xi) \odot \xi \odot \xi}.$$

Nótese cómo el factor de escalamiento exterior ξ ha desaparecido debido a la división, haciendo en este sentido que la actualización sea invariante respecto a un reescalamiento componente a componente del objetivo.

5.3.2 Algoritmos

AdaGrad

AdaGrad [9], que significa Adaptive Gradient Algorithm, corresponde a (5.3.1) con

$$\beta_1 = 0, \quad \gamma_1 = \beta_2 = \gamma_2 = 1, \quad \alpha_k = \alpha \quad \text{para todo } k \in \mathbb{N}_0.$$

Esto deja los hiperparámetros $\varepsilon > 0$ y $\alpha > 0$. Aquí $\alpha > 0$ puede entenderse como una tasa de aprendizaje “global”. Los valores por defecto en TensorFlow [20] son $\alpha = 0.001$ y $\varepsilon = 10^{-7}$. La actualización de AdaGrad se lee entonces

$$s_{k+1} = s_k + \nabla F(w_k) \odot \nabla F(w_k), \quad w_{k+1} = w_k - \alpha \nabla F(w_k) \oslash \sqrt{s_{k+1} + \varepsilon}.$$

Debido a

$$s_{k+1} = \sum_{j=0}^k \nabla F(w_j) \odot \nabla F(w_j), \tag{5.3.2}$$

el algoritmo escala, por tanto, el gradiente $\nabla F(w_k)$ en la actualización componente a componente por el inverso de la raíz cuadrada de la suma de todos los gradientes cuadrados pasados más ε . Nótese que el factor de escalamiento $(s_{k+1,i} + \varepsilon)^{-1/2}$ para la componente i será grande si los gradientes previos para esa componente fueron pequeños, y viceversa.

RMSProp

RMSProp, que significa Root Mean Squared Propagation, fue introducido por Tieleman y Hinton [33]. Corresponde a (5.3.1) con

$$\beta_1 = 0, \quad \beta_2 = 1, \quad \gamma_2 = 1 - \gamma_1 \in (0, 1), \quad \alpha_k = \alpha \quad \text{para todo } k \in \mathbb{N}_0,$$

lo que deja efectivamente los hiperparámetros $\varepsilon > 0$, $\gamma_1 \in (0, 1)$ y $\alpha > 0$. Los valores por defecto en TensorFlow [20] son $\varepsilon = 10^{-7}$, $\alpha = 0.001$ y $\gamma_1 = 0.9$. El algoritmo está dado entonces por

$$s_{k+1} = \gamma_1 s_k + (1 - \gamma_1) \nabla F(w_k) \odot \nabla F(w_k), \tag{5.3.3a}$$

$$w_{k+1} = w_k - \alpha \nabla F(w_k) \oslash \sqrt{s_{k+1} + \varepsilon}. \tag{5.3.3b}$$

El vector de escalamiento puede expresarse como

$$s_{k+1} = (1 - \gamma_1) \sum_{j=0}^k \gamma_1^j \nabla F(w_{k-j}) \odot \nabla F(w_{k-j}),$$

y corresponde a una media móvil ponderada exponencialmente sobre los gradientes cuadrados pasados. A diferencia de AdaGrad (5.3.2), donde los gradientes pasados se acumulan indefinidamente, RMSProp descuenta exponencialmente los gradientes antiguos, dando más peso a las actualizaciones recientes. Esto evita el decaimiento excesivamente rápido de las tasas de aprendizaje y permite que la convergencia a veces observada en AdaGrad, por ejemplo [24, 14], se mantenga. Los autores de Adam [18] propusieron usar como alternativa una media móvil escalada, que promedia sobre los últimos m gradientes cuadrados, para algún $m \in \mathbb{N}$ fijo. Para más detalles sobre Adadelta, véase [18, 1]. El algoritmo estándar de RMSProp no incorpora momentum, aunque esta posibilidad ya se menciona en [33]; véase también [16].

Adam

Adam [17], que significa Adaptive Moment Estimation, corresponde a (5.3.1) con

$$\beta_2 = 1 - \beta_1 \in (0, 1), \quad \gamma_2 = 1 - \gamma_1 \in (0, 1), \quad \alpha_k = \alpha \frac{\sqrt{1 - \gamma_1^{k+1}}}{1 - \beta_1^{k+1}},$$

para todo $k \in \mathbb{N}_0$, para algún $\alpha > 0$. Los valores por defecto para los parámetros restantes recomendados en [17] son $\varepsilon = 10^{-8}$, $\alpha = 0.001$, $\beta_1 = 0.9$ y $\gamma_1 = 0.999$. La actualización puede formularse como

$$u_{k+1} = \beta_1 u_k + (1 - \beta_1) \nabla F(w_k), \quad \hat{u}_{k+1} = \frac{u_{k+1}}{1 - \beta_1^{k+1}}, \quad (5.3.4a)$$

$$s_{k+1} = \gamma_1 s_k + (1 - \gamma_1) \nabla F(w_k) \odot \nabla F(w_k), \quad \hat{s}_{k+1} = \frac{s_{k+1}}{1 - \gamma_1^{k+1}}, \quad (5.3.4b)$$

$$w_{k+1} = w_k - \alpha \hat{u}_{k+1} \oslash \sqrt{\hat{s}_{k+1} + \varepsilon}. \quad (5.3.4c)$$

Comparado con RMSProp, Adam introduce dos modificaciones. Primero, debido a $\beta_1 > 0$,

$$u_{k+1} = (1 - \beta_1) \sum_{j=0}^k \beta_1^j \nabla F(w_{k-j}),$$

lo cual corresponde a un esquema de momentum (promedio móvil exponencial de los gradientes). Segundo, para contrarrestar el sesgo de inicialización proveniente de $u_0 = 0$ y $s_0 = 0$, Adam aplica una corrección de sesgo vía

$$\hat{u}_k = \frac{u_k}{1 - \beta_1^k}, \quad \hat{s}_k = \frac{s_k}{1 - \gamma_1^k}.$$

Debe notarse que existen configuraciones específicas y problemas de optimización convexa para los cuales Adam y RMSProp y Adadelta no necesariamente convergen a un minimizador, véase [28]. Los autores de [28] proponen una modificación denominada AMSGrad, que evita este problema. No obstante, Adam sigue siendo un algoritmo muy popular para el entrenamiento de redes neuronales. También nótese que, en el contexto de optimización estocástica, las demostraciones de convergencia para tales algoritmos en general todavía requieren una disminución dependiente de k de la tasa de aprendizaje “global”, como $\alpha = O(k^{-1/2})$ en (5.3.3b) y (5.3.4c).

AdamW

Una variante especialmente importante de Adam es **AdamW**, propuesta en [22], y ampliamente usada en el entrenamiento moderno de redes neuronales, en particular en modelos de lenguaje. La idea central de AdamW es desacoplar el **decaimiento de pesos** del término de gradiente adaptativo de Adam.

Recordemos primero la regularización usual por norma cuadrática. Si queremos minimizar una función objetivo F , una forma clásica de penalizar pesos grandes consiste en considerar

$$F_\lambda(w) := F(w) + \frac{\lambda}{2}\|w\|^2,$$

donde $\lambda > 0$ es un parámetro de regularización. Entonces

$$\nabla F_\lambda(w) = \nabla F(w) + \lambda w.$$

En descenso de gradiente estándar, aplicar el método a F_λ produce la actualización

$$w_{k+1} = w_k - \alpha_k(\nabla F(w_k) + \lambda w_k) = (1 - \alpha_k \lambda)w_k - \alpha_k \nabla F(w_k).$$

Por tanto, en descenso de gradiente ordinario, añadir una penalización cuadrática al objetivo es equivalente a aplicar un decaimiento multiplicativo a los pesos en cada iteración.

Sin embargo, esta equivalencia deja de ser válida para métodos adaptativos como Adam. En Adam, el gradiente se reescala componente a componente mediante una estimación de segundos momentos. Si se reemplaza $\nabla F(w_k)$ por $\nabla F(w_k) + \lambda w_k$, entonces el término de regularización también queda mezclado dentro de las estimaciones adaptativas u_k y s_k . En otras palabras, el término λw_k no actúa simplemente como un decaimiento uniforme de pesos, sino que es transformado por el preconditionamiento diagonal del algoritmo.

AdamW corrige esto desacoplando el decaimiento de pesos del gradiente usado para construir los momentos. Con la misma notación de Adam, primero se calculan los momentos usando únicamente el gradiente de la función objetivo no regularizada:

$$\begin{aligned} u_{k+1} &= \beta_1 u_k + (1 - \beta_1) \nabla F(w_k), \\ s_{k+1} &= \gamma_1 s_k + (1 - \gamma_1) \nabla F(w_k) \odot \nabla F(w_k). \end{aligned}$$

Después se aplican las correcciones de sesgo

$$\hat{u}_{k+1} = \frac{u_{k+1}}{1 - \beta_1^{k+1}}, \quad \hat{s}_{k+1} = \frac{s_{k+1}}{1 - \gamma_1^{k+1}},$$

y la actualización de AdamW toma la forma

$$w_{k+1} = w_k - \alpha_k \hat{u}_{k+1} \oslash \sqrt{\hat{s}_{k+1} + \varepsilon} - \alpha_k \lambda w_k. \quad (5.3.5)$$

Equivalentemente,

$$w_{k+1} = (1 - \alpha_k \lambda)w_k - \alpha_k \hat{u}_{k+1} \oslash \sqrt{\hat{s}_{k+1} + \varepsilon}.$$

Así, el término $-\alpha_k \lambda w_k$ produce un decaimiento directo de los pesos, mientras que el término adaptativo se calcula solamente a partir de $\nabla F(w_k)$.

Esta diferencia es conceptualmente importante. En Adam con regularización L^2 acoplada, el gradiente usado por el optimizador sería

$$\nabla F(w_k) + \lambda w_k,$$

y por tanto el decaimiento de pesos quedaría afectado por el reescalamiento adaptativo. En AdamW, en cambio, la regularización se implementa directamente como un decaimiento de los parámetros, separado de las estadísticas de primer y segundo momento. Por eso se dice que AdamW usa **decaimiento de pesos desacoplado**.

En la práctica, si $g_k = \nabla F(w_k)$, AdamW puede escribirse como

$$\begin{aligned} u_{k+1} &= \beta_1 u_k + (1 - \beta_1) g_k, \\ s_{k+1} &= \gamma_1 s_k + (1 - \gamma_1) g_k \odot g_k, \\ \hat{u}_{k+1} &= \frac{u_{k+1}}{1 - \beta_1^{k+1}}, \quad \hat{s}_{k+1} = \frac{s_{k+1}}{1 - \gamma_1^{k+1}}, \\ w_{k+1} &= (1 - \alpha_k \lambda) w_k - \alpha_k \hat{u}_{k+1} \oslash \sqrt{\hat{s}_{k+1} + \varepsilon}. \end{aligned} \quad (5.3.6)$$

Aquí $\alpha_k > 0$ es la tasa de aprendizaje, $\lambda \geq 0$ es el coeficiente de decaimiento de pesos, $\beta_1, \gamma_1 \in (0, 1)$ controlan las medias móviles exponenciales, y $\varepsilon > 0$ se introduce para estabilidad numérica.

En implementaciones modernas, el nombre **weight_decay** suele referirse precisamente al parámetro λ de AdamW. Por ejemplo, si en un código de entrenamiento se usa un optimizador de la forma

$$\text{AdamW}(\text{params}, \text{lr} = \alpha, \text{betas} = (\beta_1, \gamma_1), \text{eps} = \varepsilon, \text{weight_decay} = \lambda),$$

entonces **lr** corresponde a α , **betas** corresponde a (β_1, γ_1) , **eps** corresponde a ε , y **weight_decay** corresponde a λ .

La razón por la cual AdamW es especialmente popular en redes neuronales profundas es que combina las ventajas prácticas de Adam —escalamiento adaptativo por coordenadas y momentum— con un decaimiento de pesos más fiel a la intuición geométrica de la regularización. En particular, evita que el término de penalización λw_k sea distorsionado por las estimaciones adaptativas de segundo momento. Por esta razón, AdamW es una elección estándar en muchos entrenamientos modernos de arquitecturas tipo Transformer.

5.4 Retropropagación

En esta sección discutimos cómo aplicar métodos basados en gradiente al entrenamiento de redes neuronales feedforward. Sea $\Phi(\cdot, w)$ una red neuronal feedforward de la clase $\mathcal{N}_\sigma(d_0, \dots, d_{L+1})$ introducida en el Capítulo 3, y supongamos que la función de activación satisface $\sigma \in C^1(\mathbb{R})$. Como antes, denotamos los parámetros de la red neuronal por

$$w = ((W^{(0)}, b^{(0)}), \dots, (W^{(L)}, b^{(L)})), \quad (5.4.1)$$

con matrices de pesos $W^{(\ell)} \in \mathbb{R}^{d_{\ell+1} \times d_\ell}$ y vectores de sesgo $b^{(\ell)} \in \mathbb{R}^{d_{\ell+1}}$. Adicionalmente, fijamos una función de pérdida diferenciable $\mathcal{L} : \mathbb{R}^{d_{L+1}} \times \mathbb{R}^{d_{L+1}} \rightarrow \mathbb{R}$, por ejemplo, $\mathcal{L}(y, \hat{y}) = \|y - \hat{y}\|^2/2$, y asumimos datos dados $(x_j, y_j)_{j=1}^m \subseteq \mathbb{R}^{d_0} \times \mathbb{R}^{d_{L+1}}$. El objetivo es minimizar un riesgo empírico de la forma

$$F(w) := \frac{1}{m} \sum_{j=1}^m \mathcal{L}(\Phi(x_j, w), y_j) \quad (5.4.2)$$

como función de los parámetros w de la red neuronal. Una aplicación del paso de gradiente (5.1.2) para actualizar los parámetros requiere computar

$$\nabla F(w) = \frac{1}{m} \sum_{j=1}^m \nabla_w \mathcal{L}(\Phi(x_j, w), y_j).$$

Para métodos estocásticos, como se explicó en el Ejemplo 5.2.1, solo computamos el promedio sobre un sublot aleatorio del conjunto de datos. En cualquiera de los dos casos, necesitamos un algoritmo para determinar $\nabla_w \mathcal{L}(\Phi(x, w), y)$, es decir, los gradientes

$$\nabla_{b^{(\ell)}} \mathcal{L}(\Phi(x, w), y) \in \mathbb{R}^{d_{\ell+1}}, \quad \nabla_{W^{(\ell)}} \mathcal{L}(\Phi(x, w), y) \in \mathbb{R}^{d_{\ell+1} \times d_\ell} \quad (5.4.3)$$

para todo $\ell = 0, \dots, L$. El algoritmo de retropropagación [31] proporciona una manera eficiente de hacerlo.

5.4.1 Idea básica

Debido a la estructura composicional de las capas en una red neuronal, la función objetivo en (5.4.2) es una composición repetida de aplicaciones. El cómputo de sus derivadas requiere aplicación repetida de la regla de la cadena. Una implementación directa de esto es ineficiente debido a la aparición de cálculos redundantes. La complejidad puede reducirse significativamente almacenando y reutilizando ciertos valores intermedios. En esta sección primero mostramos esto en un contexto univariado simplificado.

Uso eficiente de la regla de la cadena

Sean $f_\ell : \mathbb{R} \rightarrow \mathbb{R}$ funciones diferenciables para $\ell = 1, \dots, L+1$. Queremos computar la derivada de

$$f_{L+1} \circ \dots \circ f_1.$$

Para $w \in \mathbb{R}$, denotemos

$$\bar{f}^{(1)} := f_1(w) \quad \text{y} \quad \bar{f}^{(\ell)} := f_\ell(\bar{f}^{(\ell-1)}) \quad \text{para } \ell = 2, \dots, L+1,$$

de modo que $\bar{f}^{(L+1)} = f_{L+1} \circ \dots \circ f_1(w)$. Por la regla de la cadena, para todo $w \in \mathbb{R}$,

$$(f_{L+1} \circ \dots \circ f_1)'(w) = f_1'(w) \cdot \prod_{\ell=2}^{L+1} f_\ell'(\bar{f}^{(\ell-1)}(w)) = \frac{\partial \bar{f}^{(L+1)}}{\partial w}.$$

Si consideramos cada evaluación de f_ℓ y f_ℓ' como una operación, entonces una implementación ingenua de esta fórmula requiere $O(L^2)$ operaciones. Si en cambio primero computamos iterativamente y almacenamos los valores $\bar{f}^{(\ell)}$ para $\ell = 1, \dots, L$, entonces el cómputo se reduce a $O(L)$ operaciones.

Pase hacia adelante y pase hacia atrás

Para redes neuronales la situación es ligeramente más complicada, pues cada capa de la red corresponde a una función que depende de la salida de la capa anterior y de sus propios parámetros. Consideramos de nuevo esto en un contexto simplificado, y suponemos que $f_1 : \mathbb{R} \rightarrow \mathbb{R}$ y $f_\ell : \mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$ son diferenciables para $\ell = 2, \dots, L+1$. Con

$$\bar{f}^{(1)} := f_1(w^{(0)}) \quad \text{y} \quad \bar{f}^{(\ell)} := f_\ell(w^{(\ell-1)}, \bar{f}^{(\ell-1)}) \quad \text{para } \ell = 2, \dots, L+1,$$

nuestro objetivo es computar las $L+1$ derivadas parciales

$$\frac{\partial \bar{f}^{(L+1)}}{\partial w^{(0)}}, \dots, \frac{\partial \bar{f}^{(L+1)}}{\partial w^{(L)}}. \quad (5.4.4)$$

Aquí $w^{(\ell)} \in \mathbb{R}$ puede interpretarse como el parámetro de la ℓ -ésima capa. La aplicación repetida de la regla de la cadena da, para $\ell = 0, \dots, L$,

$$\frac{\partial \bar{f}^{(L+1)}}{\partial w^{(\ell)}} = \frac{\partial f_{L+1}}{\partial w^{(\ell)}} \prod_{k=\ell+1}^L \underbrace{\frac{\partial f_{k+1}}{\partial f^{(k)}}}_{=: \alpha^{(k+1)}},$$

donde usamos que $\bar{f}^{(k)}$ solo depende de $w^{(\ell)}$ si $k > \ell$. Nótese que

$$\alpha^{(\ell)} = \frac{\partial \bar{f}^{(L+1)}}{\partial \bar{f}^{(\ell)}} = \alpha^{(\ell+1)} \frac{\partial f_{\ell+1}}{\partial \bar{f}^{(\ell)}} \quad \text{para todo } \ell = 1, \dots, L+1.$$

Una manera eficiente, que requiere solo $O(L)$ operaciones, de computar todas las derivadas en (5.4.4) consiste entonces en computar iterativamente las cantidades

$$\bar{f}^{(1)}, \dots, \bar{f}^{(L+1)} \quad \text{y} \quad \left(\alpha^{(L+1)}, \frac{\partial \bar{f}^{(L+1)}}{\partial w^{(L)}} \right), \dots, \left(\alpha^{(1)}, \frac{\partial \bar{f}^{(L+1)}}{\partial w^{(0)}} \right) \quad (5.4.5)$$

en este orden. El cómputo de $\bar{f}^{(1)}, \dots, \bar{f}^{(L+1)}$ se denomina el pase hacia adelante, pues la información se pasa iterativamente a través de cada capa de composición. El cómputo de la segunda parte en (5.4.5), donde la información del gradiente se construye empezando desde la función/capa más externa, se llama el pase hacia atrás.

5.4.2 Redes neuronales feedforward

Ahora volvemos al contexto (5.4.1)–(5.4.2), y aplicamos las ideas de la Sección 5.4.1 al cómputo de los gradientes (5.4.3).

Fijemos una entrada $x \in \mathbb{R}^{d_0}$ e introduzcamos la notación

$$\tilde{x}^{(1)} := W^{(0)}x + b^{(0)}, \quad (5.4.6a)$$

$$\tilde{x}^{(\ell+1)} := W^{(\ell)}\sigma(\tilde{x}^{(\ell)}) + b^{(\ell)} \quad \text{para } \ell = 1, \dots, L, \quad (5.4.6b)$$

donde la aplicación de $\sigma : \mathbb{R} \rightarrow \mathbb{R}$ a un vector es, como siempre, entendida componente a componente. En la notación de preactivaciones y activaciones introducida previamente, $x^{(\ell)} = \sigma(\tilde{x}^{(\ell)}) \in \mathbb{R}^{d_\ell}$ para $\ell = 1, \dots, L$, y $\tilde{x}^{(L+1)} = x^{(L+1)} = \Phi(x, w) \in \mathbb{R}^{d_{L+1}}$ es la salida de la red neuronal. Por tanto, los $\tilde{x}^{(\ell)}$ para $\ell = 1, \dots, L$ a veces también se denominan las preactivaciones.

En lo siguiente, adicionalmente fijamos $y \in \mathbb{R}^{d_{L+1}}$ y escribimos

$$\mathcal{L} := \mathcal{L}(\Phi(x, w), y) = \mathcal{L}(\tilde{x}^{(L+1)}, y).$$

Nótese que $\tilde{x}^{(k)}$ depende de $(W^{(\ell)}, b^{(\ell)})$ solo si $k > \ell$. Como $\tilde{x}^{(\ell+1)}$ es función de $\tilde{x}^{(\ell)}$ para cada ℓ , por aplicación repetida de la regla de la cadena

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(\ell)}} = \frac{\partial \mathcal{L}}{\partial \tilde{x}^{(L+1)}} \frac{\partial \tilde{x}^{(L+1)}}{\partial \tilde{x}^{(L)}} \cdots \frac{\partial \tilde{x}^{(\ell+2)}}{\partial \tilde{x}^{(\ell+1)}} \frac{\partial \tilde{x}^{(\ell+1)}}{\partial W_{ij}^{(\ell)}}. \quad (5.4.7)$$

Un cálculo análogo se cumple para $\partial \mathcal{L} / \partial b_j^{(\ell)}$. Para evitar cálculos innecesarios, siguiendo la idea de la Sección 5.4.1, introducimos

$$\alpha^{(\ell)} := \nabla_{\tilde{x}^{(\ell)}} \mathcal{L} \in \mathbb{R}^{d_\ell} \quad \text{para todo } \ell = 1, \dots, L+1, \quad (5.4.8)$$

y observamos que

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(\ell)}} = (\alpha^{(\ell+1)})^\top \frac{\partial \tilde{x}^{(\ell+1)}}{\partial W_{ij}^{(\ell)}}.$$

Ahora formalizamos que $\alpha^{(\ell)}$ puede computarse recursivamente para $\ell = L+1, \dots, 1$. Esto explica el nombre “retropropagación”. Como antes, $\odot$ denota el producto componente a componente.

Lema 5.4.1. *Se cumple*

$$\alpha^{(L+1)} = \nabla_{\tilde{x}^{(L+1)}} \mathcal{L}(\tilde{x}^{(L+1)}, y) \quad (5.4.9)$$

y

$$\alpha^{(\ell)} = \sigma'(\tilde{x}^{(\ell)}) \odot (W^{(\ell)})^\top \alpha^{(\ell+1)} \quad \text{para todo } \ell = L, \dots, 1. \quad (5.4.10)$$

Demostración. La ecuación (5.4.9) se cumple por definición. Para $\ell \in \{1, \dots, L\}$, por la regla de la cadena

$$\alpha^{(\ell)} = \frac{\partial \mathcal{L}}{\partial \tilde{x}^{(\ell)}} = \left(\frac{\partial \tilde{x}^{(\ell+1)}}{\partial \tilde{x}^{(\ell)}} \right)^\top \frac{\partial \mathcal{L}}{\partial \tilde{x}^{(\ell+1)}} = \left(\frac{\partial \tilde{x}^{(\ell+1)}}{\partial \tilde{x}^{(\ell)}} \right)^\top \alpha^{(\ell+1)}.$$

Por (5.4.6b), para $i \in \{1, \dots, d_{\ell+1}\}$, $j \in \{1, \dots, d_\ell\}$,

$$\left(\frac{\partial \tilde{x}^{(\ell+1)}}{\partial \tilde{x}^{(\ell)}} \right)_{ij} = \frac{\partial \tilde{x}_i^{(\ell+1)}}{\partial \tilde{x}_j^{(\ell)}} = W_{ij}^{(\ell)} \sigma'(\tilde{x}_j^{(\ell)}).$$

Así se sigue la afirmación. $\square$

Juntando todo, obtenemos fórmulas explícitas para (5.4.3).

Proposición 5.4.2. *Se cumple*

$$\nabla_{b^{(\ell)}} \mathcal{L} = \alpha^{(\ell+1)} \in \mathbb{R}^{d_{\ell+1}} \quad \text{para } \ell = 0, \dots, L,$$

y

$$\nabla_{W^{(0)}} \mathcal{L} = \alpha^{(1)} x^\top \in \mathbb{R}^{d_1 \times d_0},$$

y

$$\nabla_{W^{(\ell)}} \mathcal{L} = \alpha^{(\ell+1)} \sigma(\tilde{x}^{(\ell)})^\top \in \mathbb{R}^{d_{\ell+1} \times d_\ell} \quad \text{para } \ell = 1, \dots, L.$$

Demostración. Por (5.4.6a), para $i, k \in \{1, \dots, d_1\}$, y $j \in \{1, \dots, d_0\}$,

$$\frac{\partial \tilde{x}_k^{(1)}}{\partial b_i^{(0)}} = \delta_{ki} \quad \text{y} \quad \frac{\partial \tilde{x}_k^{(1)}}{\partial W_{ij}^{(0)}} = \delta_{ki} x_j,$$

y por (5.4.6b), para $\ell \in \{1, \dots, L\}$ y $i, k \in \{1, \dots, d_{\ell+1}\}$, y $j \in \{1, \dots, d_\ell\}$,

$$\frac{\partial \tilde{x}_k^{(\ell+1)}}{\partial b_i^{(\ell)}} = \delta_{ki} \quad \text{y} \quad \frac{\partial \tilde{x}_k^{(\ell+1)}}{\partial W_{ij}^{(\ell)}} = \delta_{ki} \sigma(\tilde{x}_j^{(\ell)}).$$

Así, con $e_i = (\delta_{ki})_{k=1}^{d_{\ell+1}}$,

$$\frac{\partial \mathcal{L}}{\partial b_i^{(\ell)}} = \left(\frac{\partial \tilde{x}^{(\ell+1)}}{\partial b_i^{(\ell)}} \right)^\top \frac{\partial \mathcal{L}}{\partial \tilde{x}^{(\ell+1)}} = e_i^\top \alpha^{(\ell+1)} = \alpha_i^{(\ell+1)} \quad \text{para } \ell \in \{0, \dots, L\},$$

y similarmente

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(0)}} = \left(\frac{\partial \tilde{x}^{(1)}}{\partial W_{ij}^{(0)}} \right)^\top \alpha^{(1)} = x_j e_i^\top \alpha^{(1)} = x_j \alpha_i^{(1)}$$

y

$$\frac{\partial \mathcal{L}}{\partial W_{ij}^{(\ell)}} = \sigma(\tilde{x}_j^{(\ell)}) \alpha_i^{(\ell+1)} \quad \text{para } \ell \in \{1, \dots, L\}.$$

Esto concluye la demostración. $\square$

El Lema 5.4.1 y la Proposición 5.4.2 motivan el Algoritmo 1, en el cual un pase hacia adelante computa $\tilde{x}^{(\ell)}$, $\ell = 1, \dots, L+1$, seguido por un pase hacia atrás para determinar los $\alpha^{(\ell)}$, $\ell = L+1, \dots, 1$, y los gradientes de $\mathcal{L}$ respecto de los parámetros de la red neuronal.

Esto muestra cómo usar optimizadores basados en gradiente de las secciones anteriores para el entrenamiento de redes neuronales.

Dos observaciones importantes están en orden. Primero, la función objetivo asociada con el entrenamiento de redes neuronales típicamente no es convexa como función de los pesos y sesgos de la red neuronal. Por tanto, los análisis de las secciones anteriores en general no serán directamente aplicables. Aun así, pueden dar intuición sobre el comportamiento de convergencia local alrededor de un minimizador local. Segundo, asumimos que la función de activación es continuamente diferenciable, lo cual no se cumple para ReLU. Usando el concepto de subgradientes, los algoritmos basados en gradientes y su análisis pueden generalizarse hasta cierto punto para acomodar también funciones objetivo no diferenciables.

Bibliography

- [1] Charu C. Aggarwal. *Neural Networks and Deep Learning: A Textbook*. Springer, 2018.
- [2] Shun-ichi Amari. Natural gradient works efficiently in learning. *Neural Computation*, 10(2):251–276, 1998.
- [3] Amir Beck. *First-Order Methods in Optimization*. SIAM, 2017.
- [4] Yoshua Bengio. Practical recommendations for gradient-based training of deep architectures. In *Neural Networks: Tricks of the Trade*, pages 437–478. Springer, 2nd edition, 2012.
- [5] L’eon Bottou, Frank E. Curtis, and Jorge Nocedal. Optimization methods for large-scale machine learning. *SIAM Review*, 60(2):223–311, 2018.
- [6] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [7] S’ebastien Bubeck. *Convex Optimization: Algorithms and Complexity*, volume 8 of *Foundations and Trends in Machine Learning*. Now Publishers, 2015.
- [8] Ovidiu Calin. *Deep Learning Architectures: A Mathematical Approach*. Springer Series in the Data Sciences. Springer, 2020. doi: 10.1007/978-3-030-36721-3.
- [9] John Duchi, Elad Hazan, and Yoram Singer. Adaptive subgradient methods for online learning and stochastic optimization. *Journal of Machine Learning Research*, 12:2121–2159, 2011.
- [10] Rick Durrett. *Probability: Theory and Examples*. Cambridge University Press, 5th edition, 2019.
- [11] Stefan Elfving, Eiji Uchibe, and Kenji Doya. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *Neural Networks*, 107:3–11, 2018.
- [12] Lawrence C. Evans. *Partial Differential Equations*, volume 19 of *Graduate Studies in Mathematics*. American Mathematical Society, 2nd edition, 2010.
- [13] Guillaume Garrigos and Robert M. Gower. Handbook of convergence theorems for (stochastic) first-order methods. *arXiv preprint arXiv:2301.11235*, 2024.
- [14] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [15] Robert M. Gower, Nicolas Loizou, Xun Qian, Alibek Sailanbayev, Egor Shulgin, and Peter Richt’arik. SGD: General analysis and improved rates. *Proceedings of the 36th International Conference on Machine Learning (ICML)*, 2019.

- [16] Alex Graves. Generating sequences with recurrent neural networks. *arXiv preprint arXiv:1308.0850*, 2013.
- [17] Geoffrey Hinton, Nitish Srivastava, and Kevin Swersky. Neural networks for machine learning, lecture 6: Overview of mini-batch gradient descent, 2012. Coursera course lecture notes.
- [18] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.
- [19] Achim Klenke. *Probability Theory: A Comprehensive Course*. Springer, 2nd edition, 2014.
- [20] Harold J. Kushner and G. George Yin. *Stochastic Approximation and Recursive Algorithms and Applications*. Springer, 2nd edition, 2003.
- [21] Yann LeCun, L'eon Bottou, Genevieve B. Orr, and Klaus-Robert M"uller. Efficient BackProp. In *Neural Networks: Tricks of the Trade*, pages 9–50. Springer, 2002.
- [22] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations (ICLR)*, 2019.
- [23] James Martens and Roger Grosse. Optimizing neural networks with Kronecker-factored approximate curvature. In *International Conference on Machine Learning (ICML)*, 2015.
- [24] H. Brendan McMahan and Matthew Streeter. Adaptive bound optimization for online convex optimization. In *Conference on Learning Theory (COLT)*, 2010.
- [25] Kevin P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.
- [26] Yurii Nesterov. *Lectures on Convex Optimization*. Springer, 2nd edition, 2018.
- [27] Philipp Petersen and Jakob Zech. *Mathematical Theory of Deep Learning*. 2026. Manuscrito en preparaci3n.
- [28] Sashank J. Reddi, Satyen Kale, and Sanjiv Kumar. On the convergence of Adam and beyond. In *International Conference on Learning Representations (ICLR)*, 2018.
- [29] Herbert Robbins and Sutton Monro. A stochastic approximation method. *The Annals of Mathematical Statistics*, 22(3):400–407, 1951.
- [30] Sebastian Ruder. An overview of gradient descent optimization algorithms. *arXiv preprint arXiv:1609.04747*, 2016.
- [31] David E. Rumelhart, Geoffrey E. Hinton, and Ronald J. Williams. Learning representations by back-propagating errors. *Nature*, 323(6088):533–536, 1986.
- [32] Galen R. Shorack. *Probability for Statisticians*. Springer, 2nd edition, 2017.
- [33] Tijmen Tieleman and Geoffrey Hinton. Lecture 6.5—RmsProp: Divide the gradient by a running average of its recent magnitude, 2012. Neural Networks for Machine Learning, Coursera.
- [34] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timoth'ee Lacroix, Baptiste Rozi'ere, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave, and Guillaume Lample. LLaMA: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

- [35] Sven-Ake Wegner. *A Mathematical Introduction to Data Science*. Springer, 2024. doi: 10.1007/978-3-031-71810-9.
- [36] Biao Zhang and Rico Sennrich. Root mean square layer normalization. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 32, 2019.